TVGL Code Analysis Report

Project: `TessellationAndVoxelizationGeometryLibrary.csproj` (net10.0, 161 C# files, ~3.8 MB of source) **Date:** 2026-07-12 (analysis of `master` @ `f18113e5`) **Method:** Every non-vendored source file was read and reviewed. The vendored Clipper2 code (`Polygon Operations\Clipper\github.com.AngusJohnson.Clipper2\`) was reviewed only for integration, not internals. Findings are grouped into (1) potential errors, (2) redundancies, (3) inefficiencies, (4) other concerns, followed by fix plans, a parallelization strategy, a Span/modern-.NET strategy, and API-design recommendations. **Full per-subsystem findings with file:line citations are in Appendices A–H** — this body is the synthesis and roadmap. No code was changed.

Executive Summary

The library is functionally rich and algorithmically ambitious, but the review found roughly **60 high-severity correctness bugs** that were verified by reading the code (not speculation), plus a long tail of medium/low issues. The dominant patterns are not exotic — they are the classic failure modes of a large, long-lived, single-team codebase:

1. **Copy-paste divergence.** Nearly every high-severity bug family comes from duplicated code drifting apart: `Vector2.IsAligned` doesn't normalize while `Vector3`'s does; the X/Y slicers lack the bounds guard the Z slicer has; the dense voxel row's `Subtract` calls `Union` while the sparse one is correct; three axis-permuted marching-cubes seed functions disagree about which cells to enqueue; the 3MF/AMF/STL/PLY/OFF writers share the same inverted `Where(string.IsNullOrWhiteSpace)` comment filter.
2. **Silent failure.** Bare `catch { }` in mesh repair and triangulation, public methods whose bodies are empty or commented out (`Create2DMedialAxis`, the four `Minkowski*` voxel methods, `CrossSectionSolid.Reverse`), `NotImplementedException` behind public API (`Save(OBJ)`, `VoxelizedSolid.Transform`, `CalculateInertiaTensor`), and lazily-deferred LINQ whose side effects are thrown away (`SimplifyMinLengthToNewPolygons` returns *unsimplified* copies). A caller cannot distinguish "worked" from "quietly did nothing" in dozens of places.
3. **Global mutable state.** `Vector3.Zero/One/UnitX...` are *mutable public static fields* (and `operator -` is implemented as `Zero - value`, so corrupting `Zero` breaks negation library-wide); `TessellatedSolidBuildOptions.Default` is a shared mutable singleton that library code writes to; `Presenter`/`Log` form an unsynchronized static service locator whose logger factory is disposed before use.
4. **Infrastructure bugs under everything.** `UpdatablePriorityQueue.Remove` can violate the heap invariant (it only sifts down) — and `UpdatePriority`, used throughout polygon and tessellation simplification, is built on it. The KD-tree hard-codes `Dimensions = 3` even for 2D points, so 2D nearest-neighbor queries use the metric $dx^2 + 2dy^2$. Comparators violate the `IComparer` contract in at least five places.
5. **Culture-sensitive IO.** ASCII STL, OFF, and PLY parse *and write* doubles with the current culture: on a German or French machine TVGL writes mesh files no tool can read, and can't read valid files. Compounded by `File.OpenWrite` (never truncates → trailing garbage when re-saving smaller files), a binary STL header that can be shorter than 80 bytes, and swapped R/B color channels between the STL reader and writer.

On the performance side, the biggest wins are not micro-optimizations but structural: $O(n^2)$ array-copy mutation loops in mesh repair, per-node heap allocations in KD-tree search, full-file regex scans in the STL reader, per-voxel virtual+locked indexer calls in voxel sweeps, and an entire `Parallel.For` (the most expensive step of voxelization) that is commented out. `Span<T>`, `ArrayPool`, `BitOperations`, and `Parallel.For` each have specific, high-leverage homes detailed in §7 and §8.

Finally, the **shipped build configuration makes ~700+ lines of native polygon boolean code dead** (all three configurations define `CLIPPER`), yet one unguarded overload still routes into the dead-and-buggy native path. Committing to Clipper2 (or fixing and testing the native path) is a decision that unblocks a lot of deletion.

Top 20 most impactful bugs (all verified; details in appendices)

#	Bug	Location	Why it matters
1	`UpdatablePriorityQueue.Remove` breaks the min-heap invariant (sift-down only); `DequeueEnqueue`/`EnqueueDequeue` leak index entries	`UpdatablePriorityQueue.cs:566-584, 278-406`	`UpdatePriority` = Remove+Enqueue underpins all polygon & mesh simplification
2	`Simplify(ts, minLength)` ignores `minLength`; loop runs until the queue is	`SimplifyTessellation.cs:132-173`; `ModifyTessellation.AddRemoveElements.cs:119-`	Public mesh-simplification entry

#	Bug	Location	Why it matters
	empty (collapses the whole mesh); companion `MergeVertexAndKill3EdgesAnd2Faces` never assigns its `keepEdge2` out-param → NRE on first successful merge	`143`	point is doubly broken
3	KD-tree always built with `Dimensions = 3`; 2D queries use metric `dx² + 2dy²`	`KDTree.cs:26-106`	Wrong nearest neighbors for all 2D uses, incl. 2D convex hull recovery
4	Culture-sensitive parse/format in ASCII STL/OFF/PLY (read **and** write)	`IOFunctions.cs:738,762,1121-1249` + writers	Broken IO on comma-decimal locales, both directions
5	`File.OpenWrite` never truncates → re-saved smaller files keep old trailing bytes	`IOFunctions.cs:1270,1351,1435`	Silent file corruption on every overwrite-save
6	All `...ToNewPolygons` simplify/complexify methods return **unmodified** copies (deferred LINQ re-enumeration)	`PolygonOperations.Simplify.cs:44-49,143-148,293-298,696-701,1124-1129`	Public API family silently does nothing
7	`AddEdges` gives every batch-added edge the same `IndexInList`	`TessellatedSolid.cs:1290`	Corrupts index-based edge removal, serialization, checksums
8	Duplicate-face checksum multiplier overflows `int` at ≥ 46,341 vertices	`TessellatedSolid.cs:806`	Silently drops faces on large meshes
9	`TessellatedSolid.Copy()` throws for any solid without primitives; faces+vertices ctor runs repair with `SameTolerance == 0`	`TessellatedSolid.cs:1474, 729-730`	Copy/slice outputs get no working repair; Copy crashes
10	`VoxelRowDense.Subtract` calls `Union`; `Intersect` with sparse operand is a no-op	`VoxelRowDense.cs:271-276, 253-259`	Wrong CSG at the row level (currently masked by forced-sparse conversion)
11	`BooleanOperation(PrimitiveSurface,…)` drops the trailing range (odd crossing counts)	`VoxelizedSolid.Advanced.cs:126-150`	`Subtract(plane)` can be a no-op; `Intersect` wrong outside bbox
12	Marching-cubes-on-layers ring buffer off-by-one: every cube reads a stale top layer and caches it	`MarchingCubesCrossSectionSolid.cs:141-148, 576-581`	Systematically wrong surfaces from cross-section solids
13	`BoundingBox` family: `Bounds`, `Center`, `Copy()`, `MoveFaceOutward`, `TransformFromUnitBox` all use unit `Directions` where dimension-scaled `Vectors` are required	`BoundingBox.cs:193, 364, 549-576, 255-264`	Almost every derived quantity of a non-unit OBB is wrong
14	`GeneralQuadric.Transform` conjugates with `M` instead of `M⁻¹`; sphere→quadric constant-term typo (`Z + Z` for `Z·Z`)	`GeneralQuadric.cs:252, 882`	Transformed/converted quadrics are wrong surfaces
15	`PolynomialSolve`: inverted `MoveNext()` guards (throws on valid input); complex	`PolynomialSolve.cs:102-106, 164-168, 181-188, 308-317`	Root-finding wrong/unusable via

#	Bug	Location	Why it matters
	`Quadratic` takes `√(−disc)`		the enumerable API; feeds eigen/quartic paths
16	`Matrix4x4.Decompose` double-negates (`-x.Negate()`) — wrong rotation for reflective matrices; `determinantBig` permutation-sign rule wrong	`Matrix4x4.cs:1483`; `inversion transpose.cs:675`	Core numerics silently wrong on pivoted/reflective inputs
17	Rotating calipers uses the *previous* rectangle's directions/offsets for side points	`MinimumEnclosure.cs:678-684`	Minimum-bounding-box side-point output wrong
18	`ConvexHull3D/4D.Create out vertexIndices` returns `[0..n-1]` (all inputs)	`ConvexHullAlgorithm.3D.cs:44`; `.4D.cs:66`	The advertised hull→input index mapping doesn't work
19	Binary STL: header can be < 80 bytes; R/B color bits swapped reader-vs-writer; inverted comment filter drops all metadata in 6 writers	`STLFileData.cs:406-408, 294-313 vs 446-452`; 6 sites	Corrupt/incorrect files in the most-used format
20	`CrossSectionSolid.Copy()` wipes the *source* solid's layers then throws NRE	`CrossSectionSolid.cs:338`	Copy destroys the original object

---

1. Potential Errors — cross-cutting view

The ~200 individual findings (Appendices A–H, §1 of each) cluster into recurring mechanisms. Knowing the mechanism tells you where the *next* bug of the same kind is hiding:

- **Assign-to-the-wrong-thing:** out-param assigned instead of local (`ARE:119`), `this.Layer2D` instead of `solid.Layer2D` (`CrossSectionSolid.cs:338`), `radii.Insert(0,…)` instead of `Insert(i,…)`, `bottomPoints.Add` instead of `topPoints.Add`, `Vertices[...]` instead of `p.Vertices[...]` in `CalculateCentroid`.
- **Boolean-logic inversions:** `if (enumerator.MoveNext()) throw` (PolynomialSolve), `Where(string.IsNullOrWhiteSpace)` (six IO writers), `!= A || != B` always-true (`IsInside.cs:986`), `&&` where `||` needed (`EltMultiply` dimension checks, Poisson-disk rejection), `if (skip && File.Exists(...)) continue` (SolidAssembly).
- **Deferred-LINQ side effects:** the `ToNewPolygons` family; `SolidAssembly.AllPartsInGlobalCoordinateSystem`; `ConvexHull3D.LineIntersection` re-sorting.
- **Empty statements as intended guards:** `if (fromNode == intersectNode) ;` (Arrangement), `if (double.IsNaN(d)) ;` (PrimitiveSurface), `;//this doesn't work IntersectRange(...)` (VoxelRowDense), empty wrap-anomaly branches in CylindricalZBuffer.
- **Contract violations in comparers and hash/equals:** `SortByIndexInList`, `NoEqualSort`, `EdgeComparer`/`NodeComparer` (both orderings return 1), `TwoDSortXFirst/YFirst`, `SphericalAnglePair` (tolerant Equals vs quantized GetHashCode), `Vector.Null` NaN sentinel breaking `Equals` reflexivity, additive `GetHashCode` in Quaternion/Matrix3x3/Matrix4x4.
- **Integer overflow at scale:** face checksum (`TS:806`, ≥46,341 vertices), 4D hull `(long)(n*n)` (`>46,340` points), voxel center accumulators (`int` for 1000^3 grids), `numVoxelsY * numVoxelsZ` products.
- **Order-of-operations / stale caches:** repair before tolerance is defined (`TS:729`), `Prismatic` ctor using `Vertices` before they exist, `startDistanceAlongDirection` computed before `stepSize` (`Slice:905-908`), lazy caches (`Torus` transforms, `Polygon.Path`, `SurfaceGroup.Faces`, primitive `faceXDir`) never invalidated by property setters or `Transform`.

Recommended verification approach: nearly every finding above is unit-testable in isolation (a 5-line repro). Before fixing, encode the top items as failing tests in `Testing\` — several fixes (heap, checksum, calipers) risk regressions if done blind.

---

2. Redundancies — cross-cutting view

2.1 Dead code that should simply be deleted (~2,500+ lines)

Item	Location	Notes
`MinimumCircleCylinderOld.cs`	Enclosure Operations (23 KB)	Namespace `TVGL.Test`, zero references, **ships in the release assembly**, contains a known bug the new file fixed
`GaussianSphere.cs`	Enclosure Operations (31 KB)	Zero references anywhere
`MakeEdgePathsTooNew` + `MakeStrandsFromEdgePaths`	`MakeEdgePaths.cs:111-272`	~160 lines, no callers, missing a guard the live copy has
`Create2DMedialAxis`	`MedialAxis2D.cs:42-281`	240-line commented body; public method returns `[]`
`SingularValueDecomposition`	`StarMathLib\svd.cs`	Internal, no callers, and broken (returns zeros)
`CreateDistanceGridBruteForce`	`MarchingCubesCrossSectionSolid.cs:195-251`	No callers
`CalculateVolumeAndCenterOLD`, `TransformToXYPlaneMaybeBetter`, `CalculateInertiaTensor` (throws), `OrderedFacesCCWAtVertexNoEdges` (throws)	MiscFunctions / VolumeCenterMoments	Dead or booby-trapped public methods
`Minkowski_sum_by_reduced_convolution_2.h`	Polygon Operations (17 KB)	C++ header checked into the C# tree
`Backup\` and `Backup1\` folders	repo root	Version control *is* the backup
Dozens of `if (false)` SSE blocks, commented algorithm alternates, unused usings, unused private helpers	All subsystems	Itemized per appendix §2

2.2 Duplication that breeds bugs (consolidate, don't just dedupe)

- **The CLIPPER triplication** (`#if CLIPPER / #elif !COMPARE / #else`) in `Boolean.cs`/`Offsetting.cs`: three bodies per public operation, one of them dead, one test-harness-only. Decision needed (see §9.4).
- **Axis-cloned algorithms:** `AllSlicesAlongX/Y/Z` (three 60-line copies with divergent bug fixes), marching-cubes `FindZ/Y/XPointFrom…` (three 120-line permutations, one already bug-diverged), dense/sparse voxel row ops, `CylindricalZBuffer` vs `ZBuffer` rasterizer (~120 lines copy-pasted).
- **Type-cloned numerics:** StarMathLib's 4–8 int/double overload copies of every routine (~70% of two 100 KB files) — collapse with generic math (`INumber<T>`, available on net10.0); `Vertex` vs `Vector3` overload pairs in MiscFunctions (75 identical lines in `AreaOf3DPolygon`); Quaternion's operator/method body copy-pastes.
- **Three worlds of linear algebra:** TVGL structs, ~90 one-line extension wrappers in `Extensions.cs`, and StarMathLib `IList<double>` routines — three names, two namespaces, four error contracts for the same operations.
- **Six copies of the IO number-reader if-ladder** and **three copies of the Open dispatch switch** (which is exactly how the OFF→PLY misrouting shipped).
- **Four point-in-polygon implementations** with different boundary semantics; **four+ point-to-segment distance routines**.

---

3. Inefficiencies — cross-cutting view

Ranked by expected real-world impact:

1. **O(n²) mesh mutation.** `RemoveVertex`/`RemoveFace`/`AddFace` each reallocate and copy the whole array, and `RemoveVertex` re-checksums *every edge*; repair loops call them per-defect (`TIR:267-317`). Fix: batch mutations (the code already does this in `SimplifyTessellation`) or move `Faces`/`Vertices` to `List<T>` with deferred compaction.
2. **KD-tree allocations.** Two `HyperRect` clones (four `double[]`) per node visited during search; per-node `List<double>` + array pair during construction. ICP multiplies this by points × iterations.
3. **ASCII STL parsing.** Every line retained in a `List<string>` and regex-scanned repeatedly (~7M regex invocations for a 1M-facet file); two `Regex` matches per facet line. Span-based parsing fixes this *and* the culture bug together.
4. **Binary IO.** A `byte[]` allocation per scalar plus LINQ `Reverse().ToArray()`; 13 allocations per binary-STL face. `stackalloc`/`BinaryPrimitives` reduce this to zero.
5. **Voxel hot loops.** Per-voxel virtual+locked indexer calls in `Draft` Y/Z and the grid enumerator; byte-at-a-time bit counting instead of `BitOperations.PopCount` over `ulong` spans; the voxelization `Parallel.For` commented out.
6. **Polygon caches.** `Polygon.InnerPolygons` re-materializes an `ImmutableArray` per access (used inside `Area`/`Perimeter`); O(n²) `RemoveAt`/`Insert(0,…)` patterns in arrangement/dedup code; O(n³) restart-the-loop unions in the native path.
7. **Per-face iterator allocation.** `TriangleFace.Edges` is a compiler-generated iterator — every `foreach` over a face's edges allocates. This is called mesh-wide in many passes; a struct enumerator or exposing `AB/BC/CA` is a library-wide win.
8. **Reflection in hot paths.** `FindBestPlanarCurve` re-scans assembly types and invokes `CreateFromPoints` via `MethodInfo.Invoke` per call.
9. **Marching cubes.** Value dictionary grows to the whole grid (eviction commented out); three arrays allocated per cube.
10. **Dijkstra with full path copies per node** (`Prismatic`), O(n·m) Delaunay2D insertion scans, double sorts, and the long tail itemized in the appendices.

---

4. Other Concerns — cross-cutting view

- **Thread safety.** The library uses `Parallel.For` internally (voxels) and invites concurrent use, but: lazy unsynchronized caches on `Edge`/`TriangleFace`/`Solid`/`Polygon`; boolean ops and slicing mutate *input* geometry (`IndexInList`, `Visited` flags); the sparse voxel row's indexer `get` is unlocked while every search mutates a shared `lastIndex`; statics (`Presenter`, `Logger`, `EqualityTolerance`, build-option singletons) are unsynchronized. **Recommendation:** declare and document a model — "solids are not thread-safe; read-only concurrent access is safe only after `X`" — then remove the per-row locks that cost every voxel indexer call, and stop mutating inputs in queries (biggest offenders: `PolygonBooleanBase.Run`, `Slice`, `TriangulateSweepLine`, `Sphere/Cone/Torus.TransformFrom3DTo2D`).
- **Debug scaffolding in production:** writes `errorPolygon*.json` to CWD (Triangulate), `cvxpoints.csv` to the **user's Desktop** (MinimumCircle), `Console.WriteLine` in hulls/ICP/Delaunay, `times.csv` under COMPARE. Remove or gate.
- **Non-determinism:** unseeded `new Random()` in Poisson-disk points, 4D hull jiggling (→ Delaunay3D), ICP — reproducibility matters precisely on the degenerate inputs where these fire. Accept seeds.
- **Tolerance zoo:** `BaseTolerance` 1e-9 (absolute), `DefaultEqualityTolerance` 1e-12, `PolygonSameTolerance` 1e-7 (relative), `OBBTolerance` 1e-5, plus dozens of inline literals (0.0001, 1.0001, 0.517, 45720000). `Solid.SameTolerance` exists but most free functions never consult it. Adopt one policy: tolerances derived from model extent (the 4D hull already does this) flowing through a context/parameter, with `Constants` values as documented fallbacks.
- **Security/robustness (IO):** AMF deserialization doesn't prohibit DTD processing (billion-laughs DoS); `TypeNameHandling.Auto` on polygon save without a binder on read; no decompression-bomb guard on 3MF/TVGLz. All cheap to close.
- **`.csproj` / packaging:** deprecated `PackageLicenseUrl` (use `PackageLicenseExpression`/`PackageLicenseFile`); description says ".NET Standard library (and a legacy portable class library)" while targeting `net10.0` only; `Copyright 2014`; all three configurations define identical `TRACE;CLIPPER` constants (so the `Series` configuration and the CLIPPER conditionals are vestigial); `Version 1.0.07.2026` uses a nonstandard scheme.

---

5. Fix Plans

A phased plan that keeps the library shippable at every step. Effort labels: S < ½ day, M = 1–3 days, L = 1–2 weeks.

Phase 0 — Correctness hotfixes (mostly one-to-five-line changes)

Write a failing test first for each; the fixes themselves are small:

1. `UpdatablePriorityQueue`: `Remove` → compare relocated node with parent, `MoveUp` or `MoveDown`; remove stale dictionary entries in `DequeueEnqueue`/`EnqueueDequeue`; fix `EnqueueRange` dictionary build. (M — with tests)
2. `SimplifyTessellation`: honor `minLength`; fix `keepEdge2` local assignment; fix double-decrement budget; populate `facesToAdd` in `SimplifyFlatPatches`. (M)
3. `TessellatedSolid`: `AddEdges + i`; `(long)NumberOfVertices * NumberOfVertices`; compute tolerance before repair in the faces+vertices ctor; guard `DefineBorders` in `Copy()` when `Primitives` is null/empty; replace the `ReplaceEdge(edge, null)` NRE path. (M)
4. IO: invariant culture everywhere (parse + format); `File.OpenWrite` → `File.Create`; pad binary STL header to 80 bytes; fix R/B bit swap; fix the six inverted comment filters; route OFF correctly in all three dispatchers; fix OBJ `values[3]` and dropped `g` groups; fix binary-PLY unknown-property skip. (M–L, mechanical but wide)
5. Polygon ops: materialize (`.ToList()`) in the five `ToNewPolygons` methods; fix `CalculateCentroid`; make `Reverse(bool)` honor its parameter; fix `IsInside.cs:986` `||`→`&&`; fix the two comparers to return 0 on equality; fix `SimplifyMinLengthToNewList` squared-vs-unsquared threshold. (M)
6. Numerics: PolynomialSolve `!MoveNext()` guards and `Sqrt(+disc)`; `determinantBig` sign via transposition count; `Decompose` single negation; `GetEigenVector2` pivot; `EltMultiply/EltDivide` `||`; make `Zero/One/Unit*/Null` fields `readonly`. (M)
7. Voxel/MC: `VoxelRowDense.Subtract` → real subtract; implement or throw on dense-sparse `Intersect`; enumerator start state; trailing-range flush in both `BooleanOperation`s and `MinkowskiSubtractOne`; `MakeFacesInCube(i, j, k-1)` ring-buffer fix; `CrossSectionSolid.Copy`/`Reverse`/`CalculateSurfaceArea`; rewrite `CalculateCenter` (long accumulators, world coords, dense/sparse-consistent sums). (M–L)
8. Enclosure/PointCloud: KD-tree dimension parameter (infer 2 for `Vector2`/`Vertex2D`); calipers stale-rectangle fix; `vertexIndices` from hull vertices; `(long)n*n`; symmetric jiggle; extreme-point tie fix; iteration caps on `MinimumSphere`/`MinimumGaussSpherePlane`. (M)
9. Primitives: `BoundingBox` — use `Vectors` in `Bounds`/`Center`/`Copy`/`MoveFaceOutward`, add scale to `TransformFromUnitBox`; `GeneralQuadric.Transform` inverse conjugation + sphere `Z*Z`; `Cone.TransformFrom2DTo3D`; `Capsule` cone-section `t`; `Torus` translation inverse; `Plane.DotCoordinate` sign; `SphericalZBuffer.XLength`; `Get3DPoint` radius double-count. (L — many small fixes across many files)
10. Misc: `SkewedLineIntersection` parentheses; `GetMinVertexDistanceAlongVector i++`; `Get3DLineValuesFromUnique` −Z branch; `radii.Insert(i,…)`; `OutputServices` logger-factory lifetime; make `EmptyPresenter2D` a true no-op. (S–M)

Phase 1 — Deletion and hygiene (low risk, high signal)

- Delete the dead-code inventory in §2.1 (verify with a solution-wide reference check per item; several were already grep-verified by this review).
- Remove Desktop/CWD file writes and `Console.WriteLine`s; route everything through `Log`.
- Fix `.csproj` metadata; remove the `Series` configuration or give it a real purpose.
- Remove `Backup\`/`Backup1\` from the repository.
- Sweep unused usings and `if (false)` blocks (an analyzer pass: enable `IDE0005`, `CA1806`, etc. as warnings).

Phase 2 — Contracts and error reporting

- One failure contract per operation family (see §9.1); eliminate `NotImplementedException`/empty bodies from the public surface (implement, throw `NotSupportedException` with a message, or delete).
- Replace silent `catch { }` in repair with a `TessellationRepairReport` returned/exposed from the build (counts of merged vertices, flipped faces, patched holes, and what failed).
- Introduce `SolidTolerances`/context threading (§4) so free functions stop mixing absolute constants with model-scale data.
- Document (and enforce with tests) the thread-safety model.

Phase 3 — Performance (guided by the existing `Benchmarking` project)

Ordered by measured-impact likelihood: mesh-mutation batching → KD-tree allocation removal → IO span parsing → voxel bit ops + re-enabled parallel voxelization → polygon cache fixes → face-edge struct enumerator → marching-cubes slab windowing. Add BenchmarkDotNet baselines before each change; details in §7–§8.

Phase 4 — API v2 (see §9)

The reorganization items (god-class splits, option objects, naming) are source-breaking; batch them into a major version with `[Obsolete]` forwarding shims for one release where practical.

---

6. Where should `Parallel` be used?

Precondition: parallelism is only safe after Phase 2's "queries don't mutate inputs" cleanup — several current bugs (e.g., `Slice` mutating `IndexInList`, boolean ops mutating vertices) make otherwise-embarrassingly-parallel work racy.

High-value, safe fan-outs (add `Parallel.For`/`ForEach`)

Site	Shape	Notes
`VoxelizedSolid.FillInFromTessellation` (`VoxelizedSolid.cs:262`)	per-k rows, disjoint writes	The `Parallel.For` is already written and commented out — the single most expensive voxelization step
Marching cubes `Generate` (`MarchingCubes.Base.cs:204`)	per-z-slab	Needs per-slab value caches merged at the end (also fixes the unbounded dictionary)
`MinimumBoundingCylinder(directions)` / `FindMinimumBoundingBox` ChanTan starts	13 independent trials	Parallel min-reduction; trivially correct
IO: building `TessellatedSolid`s for independent meshes (3MF objects, multi-solid STL, OBJ groups)	per-solid	Construction (edge-making, repair) dominates parse time
Per-face reductions: `CalculateVolumeAndCenter`, `CalculateSurfaceArea`, integrity pre-scan, `FindNonSmoothEdges`, `Transform` vertex/face loops	map/reduce with thread-local sums	Meaningful at 10^5–10^6 faces; keep a serial path below a threshold (~50k elements)
ZBuffer rasterization (`ZBuffer.cs:116`, cylindrical variant)	per-face with per-thread buffers or `Interlocked` max-compare	Faces independent except the max-store
Polygon batch ops: `SimplifyFast` over lists, Clipper path conversion, unions of disjoint groups	per-polygon	Only after `Polygon` cache thread-safety is fixed
`PrimitiveSurfaceExtensions.Voxelize` k-loop	idempotent `true` writes	Scaffolding already present, commented out

Existing parallelism to fix or remove

- **Remove** the triangulation "race" in `TessellationInspectAndRepair.cs:1269-1301` (two speculative tasks + a 1-second sleep task, unsynchronized flag, `WaitAny` NRE) — replace with sequential try-fallback or a properly cancelled race.
- **Keep** the row-parallel voxel boolean ops, but hoist the per-row LINQ allocations out of the loop and fix the unlocked sparse getter first.
- **Don't parallelize** small fixed-size numerics (2×2…4×4), per-edge repair steps with topological dependencies, or anything that walks shared mutable topology (edge paths, arrangement graphs).

---

7. Where should `Span<T>` (and friends) be used?

Concrete, high-leverage applications — all available on net10.0:

1. **Text parsing (STL/OBJ/OFF/PLY ASCII):** read lines into `ReadOnlySpan<char>`, tokenize with `MemoryExtensions.Split`, parse with `double.TryParse(span, NumberStyles.Float, CultureInfo.InvariantCulture, …)`. Kills the per-line `string.Split` arrays, the two-regex-per-facet cost, *and* the culture bug in one refactor.
2. **Binary IO:** `stackalloc byte[50]` per STL facet record (or one pooled buffer), `BinaryPrimitives.ReadSingleLittleEndian`/`WriteSingleLittleEndian` — removes ~13 allocations/face and the LINQ `Reverse().ToArray()`; makes endianness explicit.
3. **Voxel rows:** `MemoryMarshal.Cast<byte, ulong>(values)` + `BitOperations.PopCount`/`TrailingZeroCount` for `Count`, position sums, and sparse-index extraction (~8× fewer iterations, branch-free); Span-based union/intersect/subtract kernels on whole `ulong` words.

4. **Small fixed buffers → `stackalloc`/inline arrays:** GJK simplex (`Vector3[4]`, `bool[3]`), `Matrix4x4.Decompose` basis arrays, per-face index triples in `MakeVertices`/`MakeFaces`, `PointCommonToThreePlanes`' 3×3 solve, `BoundingBox.LineIntersection`'s 6-plane scratch.
5. **`ArrayPool<T>`:** marching-cubes per-layer distance grids (only 2–4 alive at once), `RemoveVertices/Faces/Edges` intermediates, KD-tree construction working arrays, LU/SVD/Cholesky scratch in StarMathLib.
6. **`CollectionsMarshal.AsSpan(list)`** for linear compaction where the code currently does O(n^2) `List.RemoveAt` loops (`MakeVerticesFromPath`, `RemoveCollinearEdgesDestructiveList`).
7. **API-level spans:** StarMathLib overloads taking `ReadOnlySpan<double>` instead of `IList<double>` (removes interface dispatch per element and enables JIT vectorization, replacing the "trust-me length" overloads); `Vector3.Coordinates` → expose `ToArray()` or a span view instead of allocating per property-get.
8. **SIMD (adjacent to Span):** `Matrix4x4` rows are exactly one `Vector256<double>` — operators, `Transpose`, `Lerp`, `FrobeniusNorm` are mechanical AVX conversions (the abandoned `if (false)` SSE scaffolding shows intent); `System.Numerics.Tensors.TensorPrimitives` for StarMathLib dot/add kernels.

---

8. Making the code easier to use — API formulation

9.1 One failure contract

Today the same conceptual operation fails four+ different ways (null return, `bool` + `out`, sentinel `Vector3.Null`/NaN matrix, exception from deep inside, silent no-op). Pick per family and enforce:

- **Queries/fits/solves:** `bool TryX(..., out result)` — already the style of `ConvexHull3D.Create`; extend to `Plane/Sphere/...TryFit(points, out surface, out error)`, `Matrix.TryInvert`, `TrySolve`.
- **IO:** one `Open`/`Save` pipeline through a single dispatcher, returning a small result object (`IOResult { Solids, Warnings, FailureReason }`) instead of the current three contracts; magic-byte sniffing so mislabeled files still open; a `CanSave(FileType)` support matrix; never advertise formats that throw `NotImplementedException`.
- **No public no-ops:** every empty/commented/`NotImplementedException` public member either works, throws `NotSupportedException` with an explanatory message, or is deleted.

9.2 Kill the global mutable state

- `Vector2/3/4` statics → `static readonly` fields (one-line fixes with outsized safety impact).
- `TessellatedSolidBuildOptions` → immutable record (`with`-based customization); library code must not write to caller-supplied options; `Default`/`Minimal` become get-only fresh instances.
- Logging/presentation: accept an `ILoggerFactory` (the packages are already referenced) and make `Presenter` an injected interface; keep the static façade as a thin default for scripts, but fix its lifetime bugs and make `Log` public so hosts can route it.

9.3 Reorganize the god classes (discoverability)

- Split `MiscFunctions` (~150 public methods across 3 partial files) into themed static classes — a concrete table is in Appendix H §5 (`DirectionalSortExtensions`, `PlanarProjection`, `AngleFunctions`, `IntersectionFunctions`, `ProximityFunctions`, `ContainmentFunctions`, `MeshTopologyFunctions`, `VolumeAndMoments`, plus a `Unique3DLine` readonly struct).
- Split `PrimitiveSurfaceExtensions` (65 KB) into border queries / tessellation factories / trimming / voxelization; promote `GetAxis`/`GetAnchor`/`GetRadius` to virtual members of `PrimitiveSurface` so new primitives can't be forgotten.
- Consider sub-namespaces (`TVGL.Polygons`, `TVGL.Voxels`, `TVGL.Numerics`, `TVGL.IO`) so IntelliSense on `TVGL.` isn't a 200-type flood; make StarMathLib `internal` except the genuinely-nD pieces (LU/eigen), funneling users to the struct types.

9.4 Decide the Clipper question

All shipped configurations define `CLIPPER`; the native boolean path is dead yet partially reachable through one unguarded overload, and it carries known bugs. Either: (a) commit to Clipper2 — delete the native path and the `#if` triplication, keep `RemoveSelfIntersections` if still needed (port it onto Clipper primitives); or (b) make the choice a *runtime* strategy (`PolygonBooleans.Engine = Native|Clipper`) and bring the native path up to tested parity. (a) is less work and removes ~1,000 lines; the compile-time define should go away in either case.

9.5 Constructors, options, and mutation clarity

- `TessellatedSolid`: replace the four constructors × seven trailing parameters with a builder or `TessellatedSolidDescription` record; make `numOfFaces:-1`/`vertices:null` conventions explicit factory methods

(`FromFaceVertexLists`, `FromTriangleSoup`).
- Separate semantics by *name*, not parameter type: `SimplifyToFaceCount(int)` vs `SimplifyByMinEdgeLength(double)` (today `Simplify(ts, 10)` silently means face-count); same for `Complexify`.
- Name the mutation convention and apply it everywhere: `Transform` (in place) vs `TransformToNew*` already exists — extend to the simplify family and fix the broken `ToNew*` copies; document what `Copy()` deep- vs shallow-copies on `Solid` and stop `CopyElementsPassedToConstructor:false` from destructively re-indexing the *source* solid.
- Primitive surfaces: freeze geometry after construction (`init` setters) or make every setter invalidate dependent caches; make query methods (`TransformFrom3DTo2D`) side-effect-free; document a single signed-distance convention ("negative = inside material") on `PrimitiveSurface.DistanceToPoint` and make all subclasses comply.
- Small foot-gun fixes: `Circle.FromRadius`/`FromRadiusSquared` factories; `Vector4.FromPoint/FromDirection` for the W convention; rename Cone `Aperture` → `ApertureSlope` (or provide `HalfAngle`); rename `ConvexHull4D.Faces`→`Tetrahedra?`/`Edge4D`→`Triangle4D`; fix `numVoxelsX` casing.

9.6 Testing & docs to lock it in

- Golden-file IO round-trip tests per format, run under `de-DE` as well as invariant culture.
- Property-based tests for geometry predicates (point-in-polygon boundary cases, hull of degenerate inputs, slicer at vertex-coincident planes).
- A behavioral test comparing `UpdatablePriorityQueue` against `PriorityQueue<TElement,TPriority>`.
- XML-doc sweep: the appendices list dozens of copy-paste doc errors ("single precision" on doubles, wrong parameter names, placeholder `<c>true</c> if XXXX`); fix alongside each touched file rather than as a big bang.

The appendices that follow contain the complete per-subsystem findings with file:line citations, severity ratings, and per-area fix lists.

Appendix A — Numerics & StarMathLib

Code Review: TVGL `Numerics/` (including `StarMathLib/`)

Scope: all 19 files under `Numerics\`. Line numbers refer to current working-tree state.

---

1. Potential Errors

High severity

1. **`PolynomialSolve` — inverted `MoveNext()` guards make the enumerable-based solvers throw on *valid* input (and silently accept empty input).** `Numerics\PolynomialSolve.cs:102-106` (`QuadraticAsTuple`), `:181-188` (`Cubic(IEnumerable)`), `:308-317` (`Quartic(IEnumerable)`):

```
if (enumerator.MoveNext()) throw new ArgumentException("Missing coefficients to solve
quadratic.");
var squaredCoeff = enumerator.Current;
```

`MoveNext()` returns `true` when an element *exists*, so any correctly-sized coefficient list throws "Missing coefficients", while an empty list sails through reading `default` from `Current`. Every guard should be `if (!enumerator.MoveNext()) throw ...`. This also breaks `GetRoots` (`PolynomialSolve.cs:55-65`) for the 3/4/5-coefficient paths (deferred until enumeration because these are iterators).

2. **`PolynomialSolve.Quadratic(ComplexNumber, ComplexNumber)` negates the discriminant before the square root.** `Numerics\PolynomialSolve.cs:164-168`:

```
var radicalTerm = linearCoeff * linearCoeff - 4 * constant;
radicalTerm = ComplexNumber.Sqrt(-radicalTerm);
```

The roots of `x² + bx + c` are `(-b ± √(b²−4c))/2`; taking `√(−(b²−4c))` multiplies the radical by `i`, giving wrong roots whenever the discriminant is not a negative real. This feeds `Cubic(ComplexNumber, …)` (`PolynomialSolve.cs:266`), which is used by `Quartic(ComplexNumber, …)` and eigenvector paths.

3. **`StarMath.SingularValueDecomposition` returns an all-zeros array of singular values.** `Numerics\StarMathLib\svd.cs:104` declares `var s = new double[sLength];` — the algorithm computes everything into `stemp` (e.g. `:131`, `:296-320`, `:519-536`), `s` is never written, and `:573` does `return s;`. All four public-facing overloads (`svd.cs:35,48,58,75`) therefore return zeros. (Mitigating: the methods are `internal` and appear to have no callers — see Redundancies #4 — but as written the API is broken.) Additionally the SVD **mutates the input matrix `A` in place** (`svd.cs:138-139,157,207`) with no doc warning, unlike `LUDecomposition` which clones.

4. **`StarMath.determinantBig(double[,])` computes the permutation sign incorrectly.** `Numerics\StarMathLib\inversion transpose.cs:675`:

```
if (permute[i] != i) result *= -1;
```

The determinant sign is `(−1)^(number of row transpositions)`, not `(−1)^(count of displaced indices)`. Example: permutation `(1,2,0)` (a 3-cycle, even, sign +1) flips the sign three times → wrong sign. Any ≥4×4 determinant that required pivoting can come back with the wrong sign. Worse, the `int[,]` twin `determinantBig` (`inversion transpose.cs:714-723`) applies **no** sign correction at all, so the two overloads disagree even with each other. It also truncates via `(int)result` (`:722`), which overflows/truncates for large determinants.

5. **`Matrix4x4.Decompose` double-negates the basis vector for left-handed matrices.** `Numerics\Matrix4x4.cs:1483`:

```
pVectorBasis[a] = -pVectorBasis[a].Negate();
```

`.Negate()` (extension, `Extensions.cs:464`) already negates; the unary `-` negates it back, so the basis vector is left unchanged while `scale` and `det` are flipped (`:1479-1484`). The reference implementation does `*pVectorBasis[a] = -(*pVectorBasis[a])`. Result: `Decompose` returns an incorrect rotation quaternion for any matrix containing a reflection/negative scale.

6. **`StarMath.GetEigenVector2` picks the *smaller* pivot — NaN eigenvectors for diagonal/triangular matrices.** `Numerics\StarMathLib\eigen.cs:856-872`:

```
if (y11.LengthSquared() > y22.LengthSquared())
{ var f = -x21 / y22; ... }   // divides by the SMALLER of the two
else
{ var f = -x12 / y11; ... }
```

The condition is inverted: when `|y11| > |y22|` you should divide by `y11` (second-row formula divides by `y22`, first-row formula by `y11`). Concretely for `A = diag(2,1)`, λ=2 gives `y11=0`, `y22=−1`; the code takes the `else` branch and computes `−x12/y11 = 0/0` → NaN eigenvector, when the correct answer `(1,0)` is available from the branch it skipped.

7. **`StarMath.EltMultiply`/`EltDivide` 2-D dimension checks use `&&` instead of `||`.** `Numerics\StarMathLib\multiply divide.cs:1578, 1592, 1606, 1621` (EltMultiply) and `:1809, 1823, 1837, 1852` (EltDivide):

```
if (A.GetLength(0) != B.GetLength(0) && A.GetLength(1) != B.GetLength(1)) throw ...
```

A mismatch in only one dimension passes the check and then either throws `IndexOutOfRangeException` deep in the loop or silently produces a wrong-shaped result (the result is even sized with mixed dims: `A.GetLength(0), B.GetLength(1)`).

Medium severity

8. **`Vector2.TransformNoTranslate` divides by zero for typical projective matrices.** `Numerics\Vector2.cs:689` (`factor = 1 / (position.X*M13 + position.Y*M23)`) and `:712` (Matrix4x4 variant: `1 / (X*M14 + Y*M24)`). A matrix flagged projective solely because `M33`/`M44` ≠ 1 has `M13=M23=0` (or `M14=M24=0`), making `factor` infinite → result `(±∞ or NaN)`. The homogeneous term (`M33`/`M44`) is dropped from the denominator; compare `Vector2.Transform` at `:644` which correctly includes `matrix.M33`.

9. **`Matrix4x4.IsNull()` never checks the fourth row for NaN.** `Numerics\Matrix4x4.cs:189-192` tests `M11…M34` for NaN but omits `M41, M42, M43, M44`. A matrix that is NaN only in the translation row (a common partial-failure mode) reports `IsNull() == false`. The zero-check part (`:193-197`) does include row 4, making the omission clearly accidental.

10. **`Matrix4x4 * scalar` and `Matrix3x3 * scalar` do not scale the homogeneous column for affine matrices.** `Numerics\Matrix4x4.cs:1982-1987`: the non-projective branch funnels through the 12-argument constructor, which hard-sets `M44 = 1` (`:327`), so `(2.0 * Matrix4x4.Identity).M44 == 1`, not 2 — i.e. the returned matrix is not `s·M`. Same for `Matrix3x3.Multiply(value1, double)` at `Numerics\Matrix3x3.cs:607-612` (`M33` stays 1). This silently breaks algebraic identities like `(sA)·x == s(A·x)` and `Lerp` built from scalar ops; `Negate` on the other hand *does* produce `−M44` (via the 16-arg ctor), so scalar `−1 * M ≠ -M`. If the renormalization is intentional it needs prominent doc; as-is it's a correctness trap.

11. **`StarMath.solveViaCramersRule3` returns `true` with infinite answers for singular systems.** `Numerics\StarMathLib\solve.cs:62-97`: `oneOverDeterminant = 1 / Determinant(a)` with no zero check; when `det == 0` and the numerator ≠ 0 the result is `±Infinity`, which passes the `double.IsNaN(x)` guards (`:69, :81, :92`) and is returned as a "successful" solve. The 2×2 twin checks `denominator == 0` properly (`solve.cs:169`).

12. **`StarMath.ExpMatrix` — Padé loop is one iteration short and the scaling is off by one.** `Numerics\StarMathLib\multiplydivide.cs:2080` `for (int k = 1; k < q; k++)` runs k=1..5; the MATLAB algorithm it cites (`expmdemo1`, comment at `:2057`) runs `k = 1:q` (6 iterations), so the 6th-order Padé term is dropped. Also `:2069` `var s = Math.Max(0, exponent)` versus the quoted `s = max(0, e + 1)` (`:2070` comment) — the scaled matrix has norm in [0.5, 1) instead of [0.25, 0.5), degrading accuracy of the fixed-order Padé approximation.

13. **`StarMath.RemoveColumn` bounds check tests against `numRows` instead of `numCols`.** `Numerics\StarMathLib\makeextract.cs:601` and `:628`:

    ```
    if ((colIndex < 0) || (colIndex >= numRows))  // should be numCols
    ```

    For a wide matrix (cols > rows) valid high column indices throw; for a tall matrix invalid indices pass the check and crash later with `IndexOutOfRangeException`. Classic copy-paste from `RemoveRow`.

14. **`Vector3.Slerp` divides by zero for parallel/antiparallel inputs.** `Numerics\Vector3.cs:768-781`: `sinOmega = Math.Sqrt(1 - dot*dot)`; when `dot = ±1`, `oneOverSinOmega = 1/0 = ∞` and the result is `∞·0 = NaN`. `Quaternion.Slerp` (`Quaternion.cs:342-347`) handles this with a lerp fallback; `Vector3.Slerp` has no such guard.

15. **`ComplexNumber.ToString()` — missing `return` makes the real-number branch dead.** `Numerics\ComplexNumber.cs:443`:

    ```
    if (IsRealNumber) string.Format(CultureInfo.CurrentCulture, "{0}", Real);
    ```

    The formatted string is discarded (statement has no effect); purely-real numbers always print as `"x + 0i"` / `"x - 0i"`.

16. **`Matrix3x3.ToString()` format string is wrong.** `Numerics\Matrix3x3.cs:744`: `"{{M11:{0} M12:{1} M12:{2}}} {{M21:{4} M22:{4} M23:{5}}} ..."` — the third label should be `M13`, `M21` uses index `{4}` (prints M22's value) instead of `{3}`, so **M21's value is never printed** and M22's is printed twice.

17. **`Vector2.IsAlignedOrReverse`/`IsAligned` compare a raw dot product against a cosine tolerance without normalizing.** `Numerics\Vector2.cs:474, 500, 512` use `this.Dot(other) >= dotTolerance` (`DotToleranceForSame ≈ cos(1°)`, `Constants.cs:117`). For non-unit vectors this is meaningless (two 0.1-length aligned vectors give dot 0.01 → "not aligned"; two long nearly-perpendicular vectors can exceed the threshold). The Vector3 versions normalize first (`Vector3.cs:517, 540, 555`) — clear copy-paste divergence. Same problem in `Vector2.IsPerpendicular` (`:523`) vs `Vector3.IsPerpendicular` (`:566`).

18. **`StarMath.IsSingular` can throw instead of returning `true`.** `Numerics\StarMathLib\inversion transpose.cs:567` ends with `A.Determinant().IsNegligible()`, but `determinantBig` → `LUDecomposition` → `findAndPivotRows` throws

`ArithmeticException` for singular matrices (`inversion transpose.cs:294-296`). So the very matrices `IsSingular` exists to detect can crash it (only the NaN path at `:672` is caught).

19. **`StarMath.GetEigenVector3` / `GetEigenVector4` return `null` for degenerate eigenspaces.** `Numerics\StarMathLib\eigen.cs:829` and `:776`: when all candidate 2×2/3×3 subsystems are singular (e.g. eigenvectors of the identity matrix, or any repeated eigenvalue with eigenspace dimension > 1), the method returns `null` with no documentation; `GetEigenVectors3/4` then hand callers arrays containing `null` entries (`eigen.cs:798-802, 728-733`).

Low severity

20. **`Vector2/3/4` NaN-`Null` sentinel breaks `Equals` reflexivity.** `Vector2.Null.Equals(Vector2.Null)` is `false` because `Equals` uses `==` on doubles (`Vector2.cs:171-174`, `Vector3.cs:158-163`, `Vector4.cs:818-821`). Since `Null` is explicitly promoted as "used in place of null" (`Vector2.cs:28-31`), a `Dictionary<Vector3,…>` or `Contains` test on Null vectors silently fails.

21. **`PolynomialSolve.Quartic(double,…)` NaN for tiny-negative discriminants.** `PolynomialSolve.cs:400-402`: `radicalTerm` negligible-but-negative (e.g. −1e−20) falls into the `else` and `Math.Sqrt(radicalTerm)` → NaN. Also `Q5`/`Q7` can be zero for degenerate quartics (four equal roots), producing `Q1/Q5` and `Q3/Q7` division by zero (`:403, :410`).

22. **`Quaternion.Slerp` uses raw `Math.Acos(cosOmega)`** (`Quaternion.cs:350`) without clamping `cosOmega` to [−1,1] (inconsistent with Vector3.Slerp which clamps).

23. **`StatisticsExtensions.NormalizedRootMeanSquareError` divides by `(xMax - xMin)`** (`StatisticCollection.cs:210`) → Infinity/NaN when all samples equal; `Mean` of empty sequence returns `0/0 = NaN` silently (`:139`); `CalcPTest` divides by `n1 - 1` (`:321`) → Infinity for single-sample groups.

24. **`Vector2.CopyTo`** — `array.Length == 0, index == 0` throws `ArgumentOutOfRangeException` rather than `ArgumentException`. Same in Vector3/Vector4.

25. **`StarMath.frexp`** (`multiply divide.cs:2100-2136`) — the `mantissa` out-param is unused by the only caller; the whole routine could be `Math.ILogB`.

---

2. Redundancies

1. **Massive int/double overload matrix in StarMathLib.** `add subtract.cs`, `multiply divide.cs`, and `make extract.cs` contain 4–8 near-identical copies of every routine (`IList<double>×IList<double>`, `int×double`, `double×int`, `int×int`; with and without explicit lengths). Examples: 12 `Add` overloads (`add subtract.cs:32-234`), 4 identical `crossProduct7` bodies (`multiply divide.cs:733-805`), 4 `KronProduct` copies (`:1945-2050`). Generic math (`INumber<T>`, .NET 7+; the project targets net10.0) or code generation would collapse ~70% of these files.

2. **Duplication between struct types and StarMathLib.** Cramer's-rule solvers exist three times: `Vector4.Solve` (`Vector4.cs:1008`), `VectorExtensions.Solve` for Matrix3x3/4x4 (`Extensions.cs:732, 748`), and `solveViaCramersRule2/3` (`solve.cs:60, 166`). Determinants: `Matrix4x4.GetDeterminant` (`Matrix4x4.cs:1063`) vs `StarMath.GetDeterminant4` (`inversion transpose.cs:629`) vs `Determinant(double[,])` (`:582`). Cross/dot products exist for `Vector3` and again for `IList<double>` (`multiply divide.cs:566+`).

3. **Duplicated operator/method bodies inside `Quaternion`.** `Multiply`/`operator *` (`Quaternion.cs:509-536` vs `:659-686`), `Divide`/`operator /` (`:562-598` vs `:720-756`), `Negate`/`operator -`, `Add`/`operator +`, `Subtract`/`operator -` are full copy-pastes instead of one delegating to the other (Vector2/3/4 delegate correctly).

4. **Dead code.**
   - `SingularValueDecomposition` (all overloads, `svd.cs:35-95`) is `internal` and has no callers in the repository — and is broken anyway.
   - `EqualityExtensions.EqualityTolerance` (`EqualityExtensions.cs:28`) is a settable static property that **nothing reads** — all methods default to the compile-time constant `Constants.DefaultEqualityTolerance`, so setting it has no effect. Misleading API.
   - `PolynomialSolve.scalfact` and `meps` (`PolynomialSolve.cs:42, 46`) are never used.
   - `IVector.Null` (`IVector.cs:10`) is a static auto-property on the interface that always returns `null`; `DefaultPoint.Null` (`IVector.cs:57`) is private and unused.

- `Vector3(Vector3 value)` copy-constructor + `Copy()` (`Vector3.cs:65, 404`) and `Vector2.Copy()` (`Vector2.cs:161`) are pointless for immutable structs.
- Large `if (false) { /* commented SSE code */ }` blocks in `Matrix4x4.cs:1570-1588, 1606-1614, 1735-1744, 1761-1768, 1785-1792, 1809-1842, 1965-1973, 1999-2006, 2023-2030` — dead scaffolding from the System.Numerics port.
- Unused usings: `PolynomialSolve.cs:17`, `StatisticCollection.cs:17`, `Vector3.cs:17,19`.

5. **Redundant overload style:** every `Vector2/3/4` static method is re-exposed as an identical extension method in `Extensions.cs` (~90 one-line wrappers, `Extensions.cs:52-836`). Doubles the API surface and XML-doc maintenance burden.

6. **`GetEigenValuesAndVectors` re-derivation:** `Extensions.cs:841-927` unpacks Matrix3x3/4x4 into 9/16 scalars to call StarMath, while `eigen.cs:68-82` re-packs `double[,]` into the same scalar lists — one canonical entry point would do.

---

3. Inefficiencies

1. **`RemoveLeadingNegIfZero` allocates on every vector `ToString`.** `Extensions.cs:35`: `string.Join(string.Empty, Enumerable.Repeat('0', numString.Length - 3))` builds a LINQ enumerator + string per component per call. A simple loop/`AsSpan().TrimEnd('0')` check avoids all of it.

2. **`Vector3.Coordinates` / `Vector4.Coordinates` allocate a new array per get** (`Vector3.cs:476`, `Vector4.cs:55`). Consider `ToArray()` methods only, or `ReadOnlySpan<double>` via inline arrays.

3. **`Vector3.Normalize` hidden costs in the hottest function of a geometry library.** `Vector3.cs:678-691`: (a) `Log.Warning` string on zero-length input — logging in a per-vertex hot path; (b) `ls.IsPracticallySame(1.0)` early-out costs a branch on every call; (c) branch structure defeats trivial inlining. Suggest a bare fast path plus a checked variant.

4. **`Matrix4x4.Decompose` allocates three arrays per call** (`Matrix4x4.cs:1365-1369`, `:1422-1427`) — replaceable with locals/`Span` since sizes are fixed at 3.

5. **StarMathLib jagged/2-D arrays and per-call allocations.**

   - Every operation allocates a fresh result; no in-place or `Span<double>` variants. `ArrayPool<double>`/`stackalloc` candidates: `LUDecomposition`'s `lastZeroIndices` — array of `List<int>` per call (`inversion transpose.cs:198-207`), `solveBig`'s LU clone (`:208`), `CholeskyDecomposition`'s full `A.Clone()` (`:413`), `SVD`'s `work/e/stemp` (`svd.cs:105-107`).
   - `GetColumn`/`SetColumn` round-trips in `GetColumns`, `RemoveRow(s)`, `RemoveColumn(s)`, `JoinMatrix*IntoVector` (`make extract.cs:244, 553, 608, 678, 1178`) copy each element twice; `Array.Copy` on the flat backing store would be one memcpy per row.
   - `multiply(double[,] A, double[,] B, …)` re-evaluates `A.GetLength(1)` in the innermost loop condition (`multiply divide.cs:1088, 1110, 1132, 1154`); also i-k-j loop order (or blocking) is more cache-friendly than current i-j-k.
   - `IList<double>` interface dispatch per element in all kernels; overloads taking `double[]`/`ReadOnlySpan<double>` would let the JIT vectorize.

6. **Missing SIMD everywhere.** `Matrix4x4` rows are 4 doubles = one `Vector256<double>`; `operator +/-/*`, `Lerp`, `Transpose`, `FrobeniusNorm` (`Matrix4x4.cs:1589-1652, 1745-1946`) are mechanical AVX candidates. Same for `Vector4.Dot/operator+` and StarMathLib `dotProduct`/`Add`. Given net10.0, `Vector256`/`TensorPrimitives` would give 2–4× with modest effort.

7. **Missed `[MethodImpl(AggressiveInlining)]`** on `Matrix3x3`/`Matrix4x4` operators and `GetDeterminant`, while trivial `Vector2.Add` wrappers are attributed. Inconsistent; the matrix ops are the ones that benefit.

8. **`StatisticsExtensions.Median`** enumerates the source twice (`StatisticCollection.cs:36-37`); quickselect uses last-element pivot with no randomization (`:97`) → $O(n^2)$ on sorted data.

9. **`Parallel.For` largely not applicable here** (matrices are 2–4 wide); only `ExpMatrix`/`LUDecomposition` on large matrices would qualify (~250×250+). Fix cache order first.

---

4. Other Concerns

1. **Mutable public static "constants" (High).** `Vector2.Zero/One/UnitX/UnitY/Null`, `Vector3.Zero/One/Null/UnitX/UnitY/UnitZ` (`Vector3.cs:376-425`), `Vector4.Zero/UnitX/.../Null` are `public static` **fields**, not `readonly`. Any code can do `Vector3.Zero = new Vector3(1,2,3)`; note `operator -` is implemented as `Zero -`

`value` (`Vector3.cs:336`), so corrupting `Zero` breaks negation globally. `Matrix3x3.Identity`/`Matrix4x4.Identity`/`Quaternion.Identity` are correctly get-only properties — the vectors should match.

2. **`Quaternion` is a fully mutable struct with public fields** (`Quaternion.cs:34-46`) while all vectors/matrices are `readonly struct`. Its `GetHashCode` is an order-insensitive **sum** of component hashes (`Quaternion.cs:829`); same additive hashing in `Matrix3x3.GetHashCode` (`Matrix3x3.cs:756-758`) and `Matrix4x4.GetHashCode` (`Matrix4x4.cs:2070-2073`). Use `HashCode.Combine`.

3. **`ComplexNumber` doesn't declare `IEquatable<ComplexNumber>`** (`ComplexNumber.cs:23`) → boxing in generic collections. `IsRealNumber` (`:394-399`) uses *relative* tolerance — scale-dependent semantics; `RealImagTolerance` (`:28`) declared but unused.

4. **Culture-sensitive formatting:** `ToString` uses `CurrentCulture` + `NumberGroupSeparator` as separator; `RemoveLeadingNegIfZero` hard-codes `"-0."` (`Extensions.cs:35-39`) — breaks in comma-decimal cultures.

5. **Inconsistent formatting defaults:** `Vector2.ToString()` uses `"G"`, `Vector3`/`Vector4` use `"F3"`; Vector3 omits `<...>` brackets. No `Parse`/`TryParse` round-trip.

6. **Thread safety:** `EqualityExtensions.EqualityTolerance`, `StarMath.MaxSvDiter` (`svd.cs:29`), `PolynomialSolve` mutable statics (`PolynomialSolve.cs:30-42`) should be `const`/`static readonly`.

7. **XML-doc drift (widespread):** "single precision" on double types (`Vector2.cs:22`, `Vector4.cs:24`); svd doc says ascending, code gives descending; `Quaternion.IsNull()` doc says "identity"; `GetColumn` error text says `numRows`; `CrossProduct` exception text wrong; `solve.cs:108-124` says "Solve2x2s"; live design doubt comments `//is M44 really 0 and not 1?` (`Matrix4x4.cs:835, 870`).

8. **Naming inconsistencies:** StarMathLib mixes public camelCase (`multiply`, `solve`, `inverse`) with PascalCase (`Add`, `Inverse`). File names contain spaces.

9. **`IsNull()` conflates NaN and all-zero for matrices** (`Matrix3x3.cs:124-133`, `Matrix4x4.cs:187-198`): a legitimately all-zero matrix reports "null".

---

5. API Usability Observations

1. **Transform convention trap:** `Vector3` has no matrix-transform operators; three same-named `Multiply` extension overloads with *different* math: `Vector3.Multiply(v, Matrix3x3)` (row-vector, `Vector3.cs:816`) vs `Matrix3x3.Multiply(matrix, v)` (column-vector, `Extensions.cs:377`) — differ only in argument order; easy to call the wrong one via extension syntax.

2. **`Vector4(double x, double y, double z)` sets `W = 1`** (`Vector4.cs:75-81`); `Coordinates => new[]{X/W, Y/W, Z/W}` (`Vector4.cs:55`) means `UnitX.Coordinates` yields `Infinity, NaN, NaN`. Point-vs-direction semantics of W deserve factory methods (`FromPoint`, `FromDirection`).

3. **NaN-sentinel defaults:** `Matrix4x4.IsIdentity(double tolerance = double.NaN)` (`Matrix4x4.cs:164`), `VarianceFromMean(…, double mean = double.NaN)` (`StatisticCollection.cs:148`) — non-discoverable; use overloads.

4. **Snap-to-affine constructors mutate user input silently** (`Matrix3x3.cs:191-197`, `Matrix4x4.cs:271-278`, fixed 1e−12 tolerance); `Matrix4x4(Matrix3x3)` (`:335-340`) hard-codes `m44 = 1` discarding the 3x3's projective scale.

5. **Inconsistent failure contracts for solve/invert:** bool+empty array (`solve.cs:36`), `Vector3.Null` (`Extensions.cs:732-753`), exception from deep LU (`inversion transpose.cs:295`), false+NaN matrix (`Matrix4x4.cs:1124`). Four contracts for one conceptual operation.

6. **`PolynomialSolve.GetRoots` coefficient ordering undocumented**; iterator-based methods defer exceptions to enumeration time.

7. **StarMathLib "trust-me" length overloads public** (e.g. `add subtract.cs:144`) — silent truncation risk; should be internal or Span-based.

8. **Three duplicate worlds** (TVGL structs, extension wrappers, StarMathLib IList) in two namespaces with different conventions and error contracts.

Top-priority fix list

1. `PolynomialSolve` inverted `MoveNext()` guards and complex `Quadratic Sqrt(-disc)`.
2. `determinantBig` permutation sign (double + int variants).
3. `Matrix4x4.Decompose` double negation.
4. `GetEigenVector2` inverted pivot condition.
5. `EltMultiply`/`EltDivide` `&&`→`||` dimension checks.
6. SVD returning zeros / delete if truly unused.
7. Make `Zero/One/Unit*/Null` statics `readonly`.

Appendix B — Polygon Operations

Code Review: TVGL `Polygon Operations\` (net10.0)

Scope: all TVGL-authored files under `Polygon Operations\`, including `PolygonBooleanOperations\` and `Clipper\ClipperPolygonOperations.cs`. The vendored Clipper2 was not deep-reviewed. Note: the csproj defines `TRACE;CLIPPER` in **all** configurations, so all `#if CLIPPER` branches are live and the TVGL-native boolean/offset code paths are dead in the shipped library.

---

1. Potential Errors

High

- **Deferred-LINQ bug: `...ToNewPolygons` methods return unmodified copies.** `PolygonOperations.Simplify.cs:44-49` (`RemoveCollinearEdgesToNewPolygons`), `:143-148` and `:293-298` (`SimplifyMinLengthToNewPolygons`), `:696-701` (`SimplifyByAreaChangeToNewPolygons`), `Simplify.cs:1124-1129` (`ComplexifyToNewPolygons`). Pattern:

  ```
  var copiedPolygons = polygons.Select(p => p.Copy(true, false));  // lazy
  SimplifyMinLength(copiedPolygons, minAllowableLength);           // iterates, mutates throwaway copies
  return copiedPolygons;                                           // re-enumeration makes FRESH, unsimplified copies
  ```

  The caller receives brand-new copies of the *original* polygons; all work discarded. (Contrast `ComplexifyToNewPolygons(double)` at `Simplify.cs:1028`, which correctly calls `.ToList()` first.)

- **`AreAllPointsInsidePolygonLines` tests the same edge repeatedly and never resets crossing parity.** `PolygonOperations.IsInside.cs:399-401`: `for (int i = lineIndex; ...) { var line = sortedLines[lineIndex]; ... }` — loop variable `i` never used to index. Also `evenNumberOfCrossings` (`:389`) declared outside per-point loop, never reset between points; parity leaks between query points. Public method effectively broken.

- **`EqualButOpposite` detection is unreachable.** `PolygonOperations.IsInside.cs:986`: `if (intersect.Relationship != NoOverlap || intersect.Relationship != Abutting || ...)` — always true; should be `&&`. `PolyRelInternal.EqualButOpposite` (`:993-999`) can never be returned; `PolygonBooleanBase.cs:73-78` handling starved.

- **`UnionPolygons(polygonsA, polygonsB)` (TVGL native path) drops non-intersecting B polygons.** `PolygonOperations.Boolean.cs:344-376` (`#elif !COMPARE` branch): `Separated` B polygons never added to result. Dead under CLIPPER but wrong if re-enabled.

- **`Polygon.Reverse(bool reverseInnerPolygons)` ignores its parameter.** `Polygon.cs:387-392`: `reverseInnerPolygons` never read. Callers: `IOFunctions.cs:613`, `IsPositive` setter (`Polygon.cs:371-379`) — hole orientation left inconsistent.

- **`Polygon.CalculateCentroid` iterates the wrong vertex list.** `Polygon.cs:571-582`: `foreach (var p in AllPolygons)` but inner loop reads `Vertices[...]` (this.Vertices), not `p.Vertices`. Wrong centroid whenever holes exist (outer counted N times, holes never).

- `CreateShallowPolygonTreesOrderedListsAndVertices` **never yields the final polygon.** `PolygonOperations.CreateTrees.cs:157-172`: last `positivePolygon` never yielded after loop. (Currently unused.)
- `TriangulateSweepLine` **failure path returns known-bad triangles.** `PolygonOperations.Triangulate.cs:264-281`: after 10 failed rotation attempts + simplification, control falls through to `AddRange(localTriangleFaceList)` (`:280`) — the failed triangle set — with no re-triangulation. Bare `catch { }` at `:248-252` swallows all exceptions.

Medium

- **Poisson-disk neighbor rejection inverted / wrong cell test.** `InternalPoints.cs:99`: `if (neighbor.Item1 || neighbor.Item2.DistanceSquared(childPt) < rSqd)` — should be `&&`; unoccupied cells test distance against `default(0,0)`. Also `:97`: `if (i == 0 && j == 0)` compares absolute grid indices instead of `i == xIndex && j == yIndex`.
- `TriangulateDelaunay` **can request unlimited internal points.** `Triangulate.cs:785-786`: `numNewVertices` can be negative; `CreateInternalPointsPoissonDisk` treats non-positive `maxPointsToReturn` as unlimited (`InternalPoints.cs:72-73`).
- `Slice2D.SliceAtLine`**: collinear-avoidance shifts computed and discarded.** `Slice2D.cs:111-113`: `positiveShift`/`negativeShift` never used; helper `ShiftLineToAvoidCollinearPoints` (`:246-259`) never called. Consequences: `SortedList` at `:137` throws `ArgumentException` on duplicate keys (line through vertex); paired loops `:156-162`/`:201-205` index out of range for odd intersection counts (tangency).
- `Polygon3D.Copy` **NRE when** `Holes == null`**.** `Polygon3D.cs:116-119`; vertices-only ctor (`:36-41`) leaves `Holes` null.
- `AllPolygonIntersectionPointsAlongVerticalLines` **index overflow.** `IsInside.cs:661`: `sortedPoints[pointIndex + 1].X` not bounds-checked; siblings use `[pointIndex]` (`:550`, `:745`).
- `AllPolygonIntersectionPointsAlongLines` **ignores its** `numSteps` **parameter.** `IsInside.cs:505-561`.
- `IsPointInsidePolygon(Polygon, bool, Vector2, out ...)` **throws on degenerate parity.** `IsInside.cs:317-321`: `ArgumentException` on tolerance-induced ambiguity; near-identical variant (`:147-153`) silently continues — inconsistent policies.
- **Empty-statement guards in** `SplitReplaceOldEdge`**.** `Arrangement.cs:163, :173`: `if (fromNode == intersectNode) ;` — dead semicolons; intended guard not applied.
- `ExtractPolygonsFromArrangementNodes` **crash / infinite-loop risk.** `Arrangement.cs:293-300`: `current.StartingEdges[0]` throws on zero-edge node; `while (current != startNode)` can cycle forever.
- `IComparer` **contract violations.** `Arrangement.cs:420-437` (`EdgeComparer`), `:438-448` (`NodeComparer`) return `1` for "equal" — `Compare(a,b)` and `Compare(b,a)` both 1. Used with `List.Sort` (`:81`, `:119`) and `BinarySearch` (`:313`) — may throw or give unreliable results.
- `SimplifyMinLengthToNewList` **compares squared lengths against unsquared threshold.** `Simplify.cs:246` enqueues `LengthSquared()` but `:256` compares against `minAllowableLength` directly. Callers (`PolygonRemoveIntersections.cs:58`, `Silhouette.cs:229`) get an effectively sqrt-scaled threshold.
- `SimplifyMinLength(paths, targetNumberOfPoints)` **double-yields on no-op.** `Simplify.cs:414-416`: missing `yield break`; polygons yielded twice.
- `SimplifyByAreaChange(Polygon, ...)` **broken for hole polygons and ignores holes.** `Simplify.cs:547-548`: budgets use signed `polygon.Area` (negative ⇒ loop exits immediately); `origArea` (`:531`) unused; only top loop enqueued (`:537`).
- `IntersectPolygons` **via Clipper computes A ∩ (B₁∪B₂∪...) instead of A₁∩A₂∩....** `Boolean.cs:608`: `BooleanViaClipper(..., Intersection, polygons.Take(1), polygons.Skip(1), ...)`. Wrong for 3+ polygons vs documented behavior (`:595-596`).
- `Complexify(List<Vector2>, double)` **inserts duplicate points.** `Simplify.cs:1104-1110`: `fraction = 0` inserts copy of `polygon[i]`; `numNewPoints` lacks the `1 +` used by Polygon overload (`:1074`).
- `HasFlippedSymmetry` **off-by-one and phase mixing.** `Basic.cs:429`: `i <= twiceNumVerts` duplicates i==0; odd shifts compare lengths against angles.
- `CreateInternalPointsRadial` **corrupts** `delta` **on wrap-around.** `InternalPoints.cs:141-151`.

- `GetMonotonicityChange` / `PartitionIntoMonotoneBoxes` **infinite loops on degenerate polygons.** `Partitioning.cs:117-123` (all-zero-length edges), `:234` (never terminates if no vertex qualifies; `:258` then indexes `Vertices[-1]`).

Low

- Stale-`Path` cache condition: `Polygon.cs:41` (`_path.Count < _vertices.Count` misses removals).
- `Polygon.Transform` updates outer bounds with inner-polygon vertices (`Polygon.cs:845-852`).
- Exact `== 0` cross-product collinearity tests (`IsInside.cs:1126-1129`).
- `PolygonEdge.FindYGivenX/FindXGivenY` return `double.MaxValue/MinValue` sentinels (`PolygonSegment.cs:280-282, 317-319`).
- `Vertex2D.this[int i]` returns `Y` for any `i != 0` (`Vertex2D.cs:53-60`).
- `DefineDeltaAngle` NaN when `maxCircleDeviation > 2|offset|` (`Offsetting.cs:120-127`).
- `Perimeter(IEnumerable<Vector2>)` NaN on empty input (`Basic.cs:69-91`).
- `IsRectangular` NaN/bare `Exception` (`Basic.cs:173, 183`).
- `MinkowskiSumConvex` — author's own comment admits uncertainty >180° (`Minkowski.cs:96-99`).

---

2. Redundancies

High

- **The entire TVGL boolean-op machinery is dead code under the shipped build.** With `CLIPPER` always defined, `Union/Intersect/Subtract/ExclusiveOr` route to `BooleanViaClipper`; `PolygonBooleanBase.cs`, `PolygonUnion.cs`, `PolygonIntersection.cs` (≈700 lines) reachable only via `RemoveSelfIntersections` (`Boolean.cs:894-902`) and the unguarded `Subtract(minuend, subtrahend, interaction, ...)` overload (`Boolean.cs:718-723` — publicly reachable with native-path bugs). Either commit to Clipper2 and remove/quarantine native path, or fix and test it.

Medium

- `#if/#elif/#else` **triplication throughout** `Boolean.cs`/`Offsetting.cs` (CLIPPER / native / COMPARE), e.g. `Boolean.cs:158-193, 233-328`, `Offsetting.cs:139-180, 227-299`. `Offset` (`:231-254`) duplicates `OffsetJust` (`:190-215`) line for line.
- **4 duplicate point-in-polygon implementations:** `IsInside.cs:207-257`, `:272-323`, `:111-158`, `:439-488` (`[Obsolete]`), with subtly different boundary/ambiguity semantics.
- **Simplify family sprawl:** `RemoveCollinearEdges*` (4+2), `SimplifyMinLength*` (9), `SimplifyByAreaChange*` (9), `SimplifyFast` (3), `Complexify*` (7) — parallel Polygon/list versions duplicate queue logic.
- **Dead/unused private code (verified):** `CreateShallowPolygonTreesOrderedListsAndVertices` (`CreateTrees.cs:157-172`); `ShiftLineToAvoidCollinearPoints` (`Slice2D.cs:246-259`); `firstAngleIsBetweenOthersCCW` (`Minkowski.cs:140-146`); `PolygonFillType` enum (`ClipperPolygonOperations.cs:25-43`); `RecursiveDelaunay` (`Triangulate.cs:995-1017` — empty loop body, stray `#endregion`); `static List<Vector2> debugPolygon` (`Triangulate.cs:827`); COMPARE harness in library (`Boolean.cs:39-130`).
- `Create2DMedialAxis` **= 240-line commented-out block returning empty list** (`MedialAxis2D.cs:42-281`). Public API that silently does nothing.
- **Large commented-out blocks** across `Polygon.cs`, `IntersectionData.cs`, `Triangulate.cs`, `PolygonRemoveIntersections.cs`, `Boolean.cs`, `Minkowski.cs:74`.
- **Unreachable code:** `Boolean.cs:634-650` (code after return), `PolygonBooleanBase.cs:233` (`if (completed == null)` on non-nullable bool).
- **17 KB C++ CGAL header** (`Minkowski_sum_by_reduced_convolution_2.h`) checked into the C# tree.

---

3. Inefficiencies

Medium

- `Polygon.InnerPolygons` **allocates a new** `ImmutableArray` **on every access** (`Polygon.cs:316-317`); accessed in hot paths (`Area :406`, `Perimeter :455`, tree recursion, boolean handlers) — O(n) copy each. Cache or expose `IReadOnlyList`.
- `Polygon.Perimeter` **re-sums inner perimeters on every get** (`Polygon.cs:455`).
- `Polygon.Reset()` **forces edge creation** (`Polygon.cs:868-869`): `foreach (var edge in Edges)` materializes all edges to reset them.

- `Area(IEnumerable<Vector2>)` **enumerates 3×** (`Basic.cs:111-117`).
- `GetWindingAngles` multiple enumeration (`IsInside.cs:53-59`).
- `SplitArrangementEdgesAtIntersections` O(n^2) (`Arrangement.cs:82-116`: `RemoveAt(0)`/`Remove(other)` in main loop); `possibleDuplicates.Insert(0, ...)` (`IsInside.cs:1211-1231`) O(n) front-inserts.
- `UnionPolygons` **native path restarts outer loop after every merge** (`Boolean.cs:266`) → worst-case O(n^3) interactions.
- **5 LINQ passes over the same intersections list** in `GetSinglePolygonRelationshipAndIntersections` (`IsInside.cs:970-1021`).
- **Per-attempt polygon rotation in TriangulateSweepLine** (`Triangulate.cs:239, 259`): mutates every vertex + `Reset()` twice per attempt, up to 10 attempts; FP drift on caller's polygon.
- `Copy` **doesn't propagate** `pathArea`/`perimeter`/`NumSigDigits` (`Polygon.cs:700-730`) — copies re-derive; `NumSigDigits` can differ, changing rounding behavior.
- **Parallelization:** `UnionPolygons`/`IntersectPolygons` over independent pairs, `SimplifyFast` map (`Boolean.cs:528-531`), `BooleanViaClipper` path conversion (`ClipperPolygonOperations.cs:151-173`) are embarrassingly parallel per polygon; no `Parallel.ForEach` anywhere in folder.
- **Span/CollectionsMarshal candidates:** `MakeVerticesFromPath` (`Polygon.cs:616-645`) per-element `RemoveAt` O(n^2); `RemoveCollinearEdgesDestructiveList` (`Simplify.cs:117-130`) same — linear compaction over a span instead.

Low

- `matchedPolygonBIndices.Count()` LINQ vs `.Count` (`IntersectionData.cs:271`).
- `AllPolygonIntersectionPointsAlong*`: `OrderBy(...).ToArray()` per step (`IsInside.cs:557, 667, 752`) — `Array.Sort` in place.
- `Polygon.Path.Any(...)` linear scan per containment query (`IsInside.cs:287`).
- No-capacity List growth (`PolygonBooleanBase.cs:193`, `IntersectionData.cs:329-343`); `.ToList()` copies of already-List returns (`PolygonBooleanBase.cs:54-55`).

---

4. Other Concerns

High

- **Thread-safety of boolean ops:** singletons `polygonUnion`/etc. (`Boolean.cs:139-147`) stateless, but `PolygonBooleanBase.Run` mutates the *input polygons*: `NumberVerticesAndGetPolygonVertexDelimiter` rewrites `vertex.IndexInList` (`PolygonBooleanBase.cs:147-161`); traversal mutates `SegmentIntersection.VisitedA/B`. Concurrent ops sharing a polygon corrupt each other. `TriangulateSweepLine` destructively rotates the input polygon (`Triangulate.cs:239`; comment `:214` mentions "threading issues").
- `Polygon` **lazy caches not thread-safe despite partial locking:** `Path`/`Area`/`PathArea`/`OrderedXVertices` lock on `_vertices`, but `Perimeter`→`Edges` mutates `_edges` unlocked (`Polygon.cs:131-136`); `Reset()` (`:860-870`) clears without lock; `Area` holds outer lock while taking nested child locks (ordering hazard); `MaxX/MinX/...` (`:469-542`) no synchronization.
- **Library writes files to CWD in production paths:** `Triangulate.cs:275`: `IO.Save(polygon, "errorPolygon" + DateTime.Now.ToOADate() + ".json")` in released CLIPPER build fallback. COMPARE-only `times.csv` and `*Fail*.json` dumps in same file.
- `NotImplementedException` **in reachable public paths:** `IntersectionData.cs:276` (reachable from public `GetPolygonInteraction`, `IsInside.cs:828`); `TriangulateDelaunay(vertexLoop/s, ...)` (`Triangulate.cs:640, 680`); `CreateInternalPointsVoronoi` (`InternalPoints.cs:118`).

Medium

- Exception swallowing: bare `catch { }` (`Triangulate.cs:248-252`); "drop last crossing edge" fallback (`Triangulate.cs:980-981`).
- Magic numbers: `scale = 45720000` (`ClipperPolygonOperations.cs:56`); miter limit `2` (`:83, :121`); `areaSimplificationFraction = 1e-5` with always-true guard (`Boolean.cs:34, 526, 603`); `100 * tolerance` (`Offsetting.cs:346`); `0.01` (`Triangulate.cs:253`); `0.25` (`Silhouette.cs:208`); mixed tolerance regimes (BaseTolerance 1e-9 abs vs PolygonSameTolerance 1e-7 rel vs DefaultEqualityTolerance 1e-12) applied inconsistently (`Basic.cs:245, 286`).
- Partial-class sprawl: `PolygonOperations` spans 15+ files; nested private types in static class (`Arrangement.cs:387-448`); invariants (who may mutate `IndexInList`) hard to reason about.
- Junk usings: `Triangulate.cs:16-20`, `Polygon.cs:20`, `Basic.cs:16`, `Simplify.cs:16`.

- Nondeterminism: unseeded `new Random()` (`InternalPoints.cs:37, 124`) vs seeded `new Random(1)` (`Triangulate.cs:226`) — inconsistent; Delaunay meshing non-reproducible.
- Copy-paste XML docs: `SegmentIntersection.cs:55-65`; `Arrangement.cs:9` header says "Minkowski.cs"; `Partitioning.cs:208`.

---

5. API Usability Observations

- **Boolean API semantics are compile-time-define-dependent** — same call runs different algorithms with different tolerance behavior; callers can't tell which they get.
- **Inconsistent tolerance conventions:** `double tolerance = double.NaN` "auto" (`Boolean.cs:209`, `Offsetting.cs:36`) vs `Constants.DefaultEqualityTolerance` default (`IsInside.cs:175`) vs `boundaryTolerance` (`:273`) vs `confidencePercentage` (`Basic.cs:170`).
- **`PolygonCollection` enum controls return *shape* but the return type is always `List<Polygon>`** — element meaning changes invisibly; `PolygonTrees` handled as `default:` (`PolygonBooleanBase.cs:101`).
- **Mutating vs copying variants hard to distinguish:** `SimplifyMinLength` vs `...ToNewPolygon(s)` vs `...ToNewList(s)`; the `ToNew*` family is currently broken. Overloads `(polygon, double)` vs `(polygon, int)` differ only in parameter type with different semantics.
- **`IEnumerable` returns with deferred side effects** — consumers enumerating twice get different results. Prefer `List<Polygon>` for anything with side effects.
- **Silent-failure public APIs:** `Create2DMedialAxis` returns `[]`; `CreateInternalPointsVoronoi` and 2 of 4 `TriangulateDelaunay` overloads throw `NotImplementedException`.
- **Naming:** `PolygonEdge` lives in `PolygonSegment.cs` (doc says "NodeLine"); `StartLine/EndLine` vs `FromPoint/ToPoint` mixed vocabularies; `AllPolygonIntersectionPointsAlongVertical` param `XValue` documented as `YValue` (`IsInside.cs:790`); `RemoveHole`/`AddInnerPolygon` asymmetry; `LargestPolygon` (signed) vs `LargestAbsAreaPolygon` (abs) with identical docs (`Basic.cs:31-43`).
- **`Polygon.IsPositive` setter** has heavyweight, partially-incorrect side effects (only reverses outer loop).
- **`CreateSilhouette` returns a single `Polygon`** (`Silhouette.cs:33-67`) — disjoint silhouette regions beyond the largest are silently discarded.

Appendix C — TessellatedSolid (mesh, repair, slicing)

Code Review: TVGL `TessellatedSolid` Subsystem

Scope: `TessellatedSolid\` plus `Solid.cs`. All findings verified by reading cited code.

File abbreviations: TS = TessellatedSolid.cs; Edge/Face/Vtx/EP = Edge.cs/TriangleFace.cs/Vertex.cs/EdgePath.cs; Opts = TessellatedSolidBuildOptions.cs; MEP = MakeEdgePaths\MakeEdgePaths.cs; TIR = ModifyTessellation\TessellationInspectAndRepair.cs; ARE = ModifyTessellation.AddRemoveElements.cs; Simp = SimplifyTessellation.cs; Cplx = ComplexifyTessellation.cs; DIV = DetermineIntermediateVertex.cs; S3PT = Single3DPolygonTriangulation.cs; Slice = Slicing\Slice.cs; CD = Slicing\ContactData.cs; Solid = Solid.cs

---

1. Potential Errors

HIGH

1. **`Simplify(ts, minLength)` ignores `minLength` entirely and will collapse the whole mesh** — `Simp:132-173`. `minLength` never referenced in loop body; with `numberOfFaces = -1` (`Simp:120-123`), `iterations` decrements from -1 and never equals 0 → loop runs until priority queue empty, collapsing *every* edge.

2. **`MergeVertexAndKill3EdgesAnd2Faces` never assigns `localKeepEdge2`; out param `keepEdge2` always null** — `ARE:119-120` assigns out param instead of local, then `ARE:143` overwrites with null local. (a) topology filter `ARE:123-126` misclassifies; (b) `Simp:164 UpdatePriority(keepEdge2, ...)` → NRE on first successful merge.

3. **`SimplifyFlatPatches` removes faces but never adds replacements** — `Simp:37 facesToAdd` never populated; new triangles only in local `newFaces` (`Simp:44,60`) and Plane primitive (`Simp:85`). `ts.AddFaces(facesToAdd)` at `Simp:89` adds nothing →

mesh gets holes.

4. `AddEdges` **assigns the same** `IndexInList` **to every added edge** — `TS:1290`: `newEdges[NumberOfEdges + i].IndexInList = NumberOfEdges;` (missing `+ i`). Affects hole repair (`TIR:1219`), `Complexify` (`Cplx:66`); breaks `RemoveEdge(int)`/`RemoveEdges` and `EdgePath.CompletePostSerialization` (`EP:480-481`).

5. `RemoveEdge`/`RemoveReferencesToEdge` **crashes via** `ReplaceEdge(edge, null)` — `TS:1377-1380` → `Face:493` dereferences `newEdge` → NRE for any removed edge with an owned/other face. Public `RemoveEdge(int)` (`TS:1309`) unusable on connected edges.

6. **Faces+vertices constructor runs repair with** `SameTolerance == 0` — `TS:729-730`: repair called *before* `DefineAxisAlignedBoundingBoxAndTolerance()`. `SameTolerance` defaults 0 (`Solid:254`); negligible-area detection uses `0*0` (`TIR:266,364`); crack-stitching `maxTolerance = 100*0 = 0` (`TIR:859`) — can never match, for solids built via this ctor (used by `Copy()` and all `Slice` outputs). Other ctors compute tolerance first (`TS:143,208`).

7. **Int overflow in duplicate-face checksum multipliers** — `TS:806`: `NumberOfVertices * NumberOfVertices` computed in 32-bit int before widening. Overflows at ≥46,341 vertices → false duplicate detection, silently dropped faces; guard at `TS:799` only kicks in at ~2M. Should be `(long)NumberOfVertices * NumberOfVertices`.

8. **Race + null deref in hole triangulation task juggling** — `TIR:1269-1301`. Three tasks race incl. `Task.Run(() => Thread.Sleep(1000)) //why?`. `Task.WaitAny` returns first to *complete*, not succeed: if task 0 throws (caught, `triangleFaceList1` null) and completes first, `TIR:1301` NREs. Shared `success` flag unsynchronized and never read. If both triangulations >1s, sleep task "wins" and hole silently skipped.

9. `Complexify` **inserts NaN/Null vertices when an edge spans two primitives** — two-primitive `DetermineIntermediateVertexPosition` returns `Vector3.Null` (`DIV:165-168`, via `DIV:38`). In `Cplx:100-115` NaN deviation fails break check (NaN comparisons false) and `new Vertex(c.mpt)` (`Cplx:115`) inserts NaN vertex.

10. `PropagateFixToNegligibleFaces` **ignores bool result of** `CollapseEdgeAndKill2MoreEdgesAnd2Faces` — `TIR:274`. On refusal (`ARE:127-139` returns false, `removeEdges = null`), still calls `ts.RemoveVertex` (corrupt mesh) and `ts.RemoveEdges(null)` → NRE at `TS:1330`. Also `OtherFace == null` (open mesh) → NRE at `TS:1208`.

11. `Copy()` **throws for any solid without primitives** — `TS:1474` unconditionally calls `DefineBorders(copy)`; first statement `solid.Primitives.First().Borders` (`TIR:1370`) throws. Also breaks `TransformToNewSolid` (`TS:1596-1601`), `SetToOriginAndSquareToNewSolid` (`TS:1517-1521`).

12. **Cross-sections by** `numSlices` **alone produce NaN offsets** — `Slice:905-908`: `startDistanceAlongDirection` computed *before* `stepSize` derived from `numSlices`; NaN slice distances, all layers empty. Also unconditionally overwrites caller-supplied start (NaN check commented out `Slice:903-904`).

MEDIUM

13. `MakeContactDataForEachSolid` **tests** `face.B` **twice, never** `face.C` — `Slice:324-326`.
14. `Transform` **rotates inertia tensor incorrectly** — `TS:1581-1585`: $I \cdot R$ instead of $R \cdot I \cdot R^T$; comment says "I'm not sure this is right". Translation effects ignored.
15. **Mutation methods never invalidate cached** `_volume`/`_surfaceArea`/`_center`/`_inertiaTensor` — `TS:1157-1236, 1015-1122, 1280-1294` leave `Solid:89-155` caches stale. Post-repair `ts.Volume` reports pre-repair value (`TIR:359`).
16. `Vertex.DefineCurvature` **inverted for mixed edges** — `Vtx:179-190`: `Any(e => e.Curvature != Convex)` → Concave; final else unreachable. Compare correct `All`-based `Face:457-466`.
17. `Loop` **ctor checks field before assignment** — `CD:483-486`: reads `IsClosed` (always false) instead of `isClosed` param.
18. `GetSliceContactData` **can never return** `false` — `Slice:135-155` single `return true`; "plane doesn't cut" handling at `Slice:36-41, 81-86, 111-117` is dead; surfaces as exception from `DivideUpFaces` (`Slice:417`).
19. `AllSlicesAlongX/Y` **index past end of** `sortedVertices` — `Slice:1326-1327`, `:1385-1386`; Z variant guards (`Slice:1444`) but its `break` leaves remaining `loopsAlongZ[step]` entries **null**.
20. `DivideUpFaces` **starting-edge search throws even when a good edge is found** — `Slice:485-491` (boundary condition on k).
21. `EdgePath.IndexOf`/`Contains((Edge,bool))` **throws on missing edge** — `EP:304-306`: `DirectionList[-1]`.
22. `EdgePath.AddBegin(Edge)` **corrupts/throws on empty path** — `EP:273-278`.
23. `Complexify` **stuffs a sequence counter into** `Edge.EdgeReference` — `Cplx:130,138,149`; since >0, `AddEdge(s)` (`TS:1270,1289`) never recompute → bogus checksums break `TIR:968`, `Simp:64`.

24. `FlipEdge` **NREs on boundary edges; stale primitive face refs** — `ARE:35` no null check on `oldOtherFace`; `ARE:81-85` never removes old faces from `primitive.Faces`.
25. `DefineBorderSegments` **dereferences before its own null check** — `TIR:1483-1486`.
26. `MakeEdges` **inconsistent null guard on** `SingleSidedEdgeData` — `TIR:612-613` vs `TIR:639`.
27. `TriangulateRecurse` **dequeues without count check** — `S3PT:479-481` → `InvalidOperationException`.
28. **StreamRead color array overflow after degenerate-face skipping** — `TS:483-498` vs `TS:532-552`: run-length encodes original face count; `colors[k++]` sized to shrunk `ts.Faces.Length` (`TS:533`).
29. `Simplify` **iteration counting removes ~half the requested faces** — `Simp:143,151,156`: budget decremented per attempt AND per success.

LOW

30. `MakeVertices` decimal-rounding loop can throw for degenerate bounds — `TS:938-940` (`numDecimalPoints` past 15).
31. `vertsPerFace` re-enumerated for count — `TS:141-142`.
32. `RemoveVertices`/`RemoveFaces`/`RemoveEdges` mis-handle duplicate indices — `TS:1112-1119, 1228-1234, 1349-1355`; no dedup at callers.
33. `Edge.DefineInternalEdgeAngle` div-by-zero (`Edge:336,347`, `numNeighbors == 0` → NaN).
34. `StreamWrite` color block indexes `Faces[0]` with no empty-mesh guard — `TS:313-327`.
35. `StreamRead` NREs if `NumberOfPrimitives` absent — `TS:560` vs `TS:408`.
36. `GetFacePatchesBetweenBorderEdges` iterates `face.Edges` (may contain nulls) not `NonNullEdges` — `TIR:1193-1196`.
37. `SortByIndexInList` violates comparer contract on equal indices — `Comparators.cs:33-38`; `SortedSet.Remove` in `Simp:139-141` can silently fail (likely given #4).
38. `FindBestEdgePathPair` can index `edgePaths[-1]` — `MEP:309-326` when all pairs score MaxValue.

---

2. Redundancies

1. **HIGH — Three near-identical 60-line slicers plus a general one**: `AllSlicesAlongX/Y/Z` (`Slice:1296-1344, 1354-1403, 1413-1465`) + `GetUniformlySpacedCrossSections` (`Slice:890-958`) duplicate the same sweep with divergent bugfixes (Z has bounds guard, X/Y don't; `0.001*` vs `/10.0` offsets — `Slice:942` vs `:1327`). Also duplicated 30-line boundary-edge region in `NewFace` (`Slice:604-634` vs `:661-691`).
2. **HIGH —** `MakeEdgePathsTooNew` **+** `MakeStrandsFromEdgePaths` **are dead code** (~160 lines, `MEP:111-272`); no references outside own file; dead copy lacks `ep1 != ep2` guard (`MEP:64` vs `:159`).
3. **MEDIUM — Duplicated edge-matching logic**: `TS:805-927` checksum machinery (incl. unused private helpers `TS:871-879, 892-912, 924-927`) vs `TIR:365-414`; three separate walk-edges/checksum/complete-or-create loops (`TIR:1302-1356`, `Simp:61-83`, `TS:831-857`).
4. **MEDIUM — Dead/commented code**: 25-line commented `StartArray` branch (`TS:446-472`); 50-line commented quadric derivation (`DIV:91-144`); `#if PRESENT` blocks (`TIR:765-773, 846-853`); no-op `if (secondVertex == thirdVertex) secondVertex = thirdVertex;` (`S3PT:459,469`).
5. **LOW** — `SetNegligibleAreaFaceNormals` (`TIR:664-686`) dead private method.
6. **LOW —** `DetermineIntermediateVertexPosition` **called twice back-to-back** — `ARE:179-184`: first result immediately overwritten by two-primitive overload (which returns Null — see Errors #9).
7. **LOW —** `RemoveFaces(IEnumerable)` **sorts twice** — `TS:1208-1209` + `TS:1225`.

---

3. Inefficiencies

1. **HIGH — O(n²) incremental element mutation in repair paths**: every `RemoveVertex`/`RemoveFace`/`AddFace` reallocates and copies full array (`TS:1002-1008, 1070-1085, 1142-1151`); `RemoveVertex` re-checksums *every edge* (`TS:1084,1121,1127-1132`). `PropagateFixToNegligibleFaces` (`TIR:267-317`) does this per defect in a while loop → O(k·(V+E)). Batch removals (as `Simp:170-172` does) or use List.
2. **HIGH — Dead debug allocation in** `Complexify` — `Cplx:91`: allocates 2 Vector3 arrays per edge per call; only referenced by commented-out presenter code.
3. **MEDIUM — Linear search in straddle-loop tracing** — `Slice:438-446`: `FirstOrDefault` inside do/while → O(n²) per slice; `straddleEdgesDict` (`Slice:387`) exists but maps to raw Edge.
4. **MEDIUM —** `GetNeighborsPreEdgeCreation` **scans pair lists linearly** (`TIR:1016-1038`) — O(n·m) on damaged meshes.
5. **MEDIUM — LINQ in hot loops**: `Intersect(...).Any()` in O(n²) pair loop (`MEP:302-305`, per-pair hash sets); `Faces.All(...)` per `StreamWrite` (`TS:313`); iterator allocations in per-face loops (`TIR:368-413`, `TS:288-291`);

`Edges.Any/All` chains (`Face:459-465`).

6. **MEDIUM — Misused parallelism**: only concurrency is the broken triangulation race (`TIR:1269-1298`). Genuinely parallelizable per-face passes are serial: `CalculateSurfaceArea` (`TS:1633`), integrity face scan (`TIR:368-414`), `Transform` loops (`TS:1550-1577`), `FindNonSmoothEdges` (`TIR:1153-1167`). Dominant costs at 10^5–10^6 faces.
7. **LOW — Span/ArrayPool candidates**: `MakeVertices` per-face `List<int>` (`TS:950`) → stack Span of 3; `MakeFaces` per-face `orderedIndices` lists (`TS:814`) — the 3-value sort helper (`TS:892`) written for this is unused; Remove* intermediate arrays → ArrayPool.
8. **LOW — Lazy values recomputed on every access while Undefined**: `Vertex.Curvature`, `TriangleFace.Curvature` (`Vtx:159-167`, `Face:439-447`).
9. **LOW — `GetSliceContactData` copies distances list twice** (`Slice:145-147`, `:382-383`).

---

4. Other Concerns

1. **HIGH — Mutable static build-option singletons + option mutation**: `TessellatedSolidBuildOptions.Default`/`Minimal` (`Opts:10,18`) shared mutable with public setters; `CompleteBuildOptions` *writes to caller's options object* (`TIR:115-119`). Global/cross-thread behavior mutation.
2. **HIGH — `IndexInList` fragility is systemic**: `Edge.EdgeReference` checksums (`Edge:459-477`), primitive `FaceIndices` (`PrimitiveSurface.cs:268-276` — only refreshed when it *shrinks*, so removals leave stale indices used by `Copy()` `TS:1434`), `EdgePath` serialization (`EP:470,481`), `Slice`'s `distancesToPlane[edge.From.IndexInList]` (`Slice:390-391`). Findings 1.4/1.23 show indices do drift. No invariant checker outside debug-only `TIR:457-539`.
3. **MEDIUM — Silent repairs and swallowed exceptions**: every repair stage wrapped in `catch { }` or log-only (`TIR:162-251`; bare catches `TIR:180-183, 206-209`). No summary of changes; `out removed*` lists discarded by ctors (`TS:163,211,729`).
4. **MEDIUM — Thread safety of lazy mutable state**: `Edge.Length/Vector/InternalAngle/Normal` (`Edge:104-256`), `Face.Normal/Center/Area` (`Face:236-405`), `Solid.Volume/...` (`Solid:65-155`) unsynchronized lazy fields. `Slice` header admits conflict (`Slice:11-15`); concurrent slicing mutates `IndexInList` on shared vertices (`Slice:510,522,548`).
5. **MEDIUM — Serialization**: malformed primitive entries `break` mid-object (`TS:421-444`); `Type.GetType("TVGL." + primitiveString)` (`TS:433`) fragile; `SurfaceArea` silent no-op setter (`Solid:120`).
6. **MEDIUM — Memory footprint**: face = class w/ 3 vertex + 3 edge refs + caches (~100+ B); vertex carries two List<> allocations (`Vtx:53-54`). Struct-of-arrays layout would cut memory several-fold and improve locality.
7. **LOW — `EdgePath` claims `IsReadOnly => true`** (`EP:176`) while mutable through `RemoveAt`/`Remove`/`Clear`; `Add`/`Insert` throw (`EP:324-351`) — violates IList expectations.

---

5. API Usability Observations

1. **Constructor overload sprawl** — 4 public ctors (`TS:136, 179, 201, 650`) with trailing `(colors, buildOptions, units, name, filename, comments, language)`; `numOfFaces: -1` sentinel; colors recycled modulo; `vertices: null` = "derive from faces". Builder or description object would clarify.
2. **Inconsistent `DuplicateFaceCheck` default** — `Opts:102` false, but indexed ctor treats *null options* as true (`TS:209`); verts-per-face ctor ignores option (`TS:162`).
3. **Repair entry points hard to discover, inconsistently gated** — implicit in ctors; post-hoc handles: `ts.Errors` (property named Errors is the repair engine, `TS:106`), `MakeEdgesIfNonExistent` (`TS:736-749`). Some option combos throw (`TIR:214-215`), others silently auto-correct (`TIR:115-119`).
4. **Mutation vs immutability confusion** — `CopyElementsPassedToConstructor=false` destructively reindexes the *input solid's* elements (`TS:685-691, 716-727`); `Transform` destructive vs `TransformToNewSolid`; `TurnModelInsideOut` half-updates caches (`TS:1608-1618` negates `_volume` not `_center`); public setters on `Faces`/`Vertices`/counts (`TS:57-99`).
5. **`Simplify`/`Complexify` overload semantics collide** — `(ts, int)` vs `(ts, double)` (`Simp:110-123`, `Cplx:39-52`): integer-typed call silently picks face-count semantics. Named methods safer — esp. given min-length isn't implemented (finding 1.1).
6. **`Errors == null` overloaded** — means both "checked and clean" and "never checked" (`TIR:441-450`, `TS:739`).

Appendix D — Enclosure Operations & PointCloud

Code Review: TVGL Enclosure Operations & PointCloud

All findings verified by reading the source. Citations relative to `TessellationAndVoxelizationGeometryLibrary\`.

---

1. Potential Errors

High

1. **All KD-trees are built with `Dimensions = 3`, even for 2D points — nearest-neighbor results are wrong for 2D.** Every factory passes 3: `PointCloud\NearestNeighbor\KDTree.cs:26` (`new KDTree<Vector2>(3, …)`), `:42` (Vertex2D), `:58`, `:73-74`, `:89-90`, `:105-106`. `Vector2`'s indexer returns Y for any index ≥ 1 (`Numerics\Vector2.cs:65-72`), so `StraightLineDistanceSquared` (`KDTree.cs:480-489`) computes $dx^2 + 2 \cdot dy^2$ — a metric that *orders points differently* than Euclidean. Radius queries filter with the inflated metric. Directly affects `ConvexHullAlgorithm.2D.cs:90/108` (`CreateConvexHullMaximal` uses `kdTree.FindNearest(midPoint, radius)`), which can miss boundary points it was written to recover.

2. **`GetDistanceToExtremePoints` adds max-side ties to `bottomPoints`.** `MinimumEnclosure.cs:479-480`: `if (distance.IsPracticallySame(maxD, ...)) bottomPoints.Add(point);` — should be `topPoints.Add(point)`. Consumed by `BoundingRectangleAlong` (`:87-95`).

3. **`ConvexHull3D.Create(..., out vertexIndices)` returns indices of *all* input points, not hull vertices.** `ConvexHullAlgorithm.3D.cs:37-44`: `vertexIndices = vertices.Select(v => v.IndexInList).ToList()` = always `[0..n-1]`. Should project `convexHull.Vertices`. Identical bug in 4D: `ConvexHullAlgorithm.4D.cs:66`.

4. **Rotating calipers: side points computed with the *previous* best rectangle's directions/offsets.** `MinimumEnclosure.cs:678-684`: `FindSidePoints` called with `bestRectangle.Direction1/2`/`Offsets(i)` *before* `bestRectangle` is replaced. First improvement compares against the dummy rectangle's ±∞ offsets (`:637-638`); later improvements use stale directions. Propagates into `FindOBBAlongDirection`'s `PointsOnFaces` (`:875-883`).

5. **`ConvexHull4D.JigglePointsAndTryAgain`: int overflow and one-sided jiggle.** `:211`: `var nSqd = (long)(n * n);` overflows for n > 46,340 (correct form at `:105`), corrupting face-ID hashing. `:196-197`: `p.X + 2*stepSize* (random.NextDouble() - 1)` gives perturbation in $[-2 \cdot step, 0)$ — every coordinate shifted negatively (should be `- 0.5`).

6. **2D convex hull extreme-point tie-breakers reference the wrong extreme.** `ConvexHullAlgorithm.2D.cs:170` (`x > points[maxXIndex].X` should be `minYIndex`) and `:182` (should be `maxYIndex`). With AABB ties, wrong seed extreme; CCW-dedup at `:212-223` assumes correct tie-breaking.

7. **ICP: `GetAngles` loops over the wrong collection size.** `IterativeClosestPoint.cs:421-427`: bound is `startNormals.Count` but indexes target-cloud-sized lists → IndexOutOfRange or silent truncation for differing cloud sizes.

8. **ICP: division by zero for `maxIterations < 50`.** `IterativeClosestPoint.cs:145`: `% ((int)(0.02 * maxIterations))` — modulus 0.

Medium

9. **`GetExtremaOnAABB` skips every other point.** `ConvexHullAlgorithm.3D.cs:599` and `.4D.cs:675`: `for (int i = 1; i < n; i += 2)`. Half the input never participates in the Akl-Toussaint extrema search; seed simplex degraded; 3D degenerate-case detection (`:108-128`) fed by incomplete extrema.

10. **`MinimumSphere` / `MinimumGaussSpherePlane` can loop forever on oscillation.** Stall counter only increments on consecutive same index (`MinimumSphere.cs:73-74`, `MinimumGaussSpherePlane.cs:68-69`); a 2-cycle (A,B,A,B from the `1.0001` slop at `MinimumSphere.cs:356-357`) resets it; no absolute cap (contrast `MinimumCircleCylinder.cs:49` maxIterations = 1000).

11. **`ConvexHull4D.Create` tolerance ignores the W extent** (`.4D.cs:28-62`). For `Delaunay3D` ($W = x^2+y^2+z^2$, `Delaunay3D.cs:400,410`) the jiggle step is badly under-scaled.

12. **`Delaunay2D.Create`: `IndexInList` off-by-one in two branches.** `Delaunay2D.cs:148, :151`: `delaunayVertices[i++] = new Vertex(…, i);` — side effect before argument evaluation → IndexInList = position + 1 for IVector3D and plain IVector2D inputs.

13. **`Delaunay2D` degenerate handling.** `Circle.CreateFrom3Points` success flag ignored (`:263`, super-circle `:172`) → collinear/dupe points poison containment; duplicate points silently dropped from mesh while still in `Vertices` (strict `<` at `:305-308`).

14. `Delaunay3D.Create(…, reuseInputVertices: true)` **never reuses input vertices.** `Delaunay3D.cs:283-286`: result immediately overwritten — dead assignment contradicting docs. `CreateInner`'s `points is IList<Vector3>` check (`:394`) can never be true (no variance over value types) — dead fast path.
15. **GJK degenerate path can throw.** `ConvexHullGJK.cs:306-308`: `dotTotal == 2` case has empty `else { // ERROR; }` falling to `throw new Exception("This should never happen")` (`:462`). `hff1` (`:589-592`) has needless cancellation.
16. **KD-tree: empty input crashes.** `KDTree.cs:274`: `GetNullObject(OriginalPoints[0])` — no Count == 0 guard.
17. `MinimumCircle` **fallbacks.** `FirstCircle` ignores `CreateFrom3Points` bool (`MinimumCircleCylinder.cs:142`); `FindCircle`'s `if (circle.Radius == 0.0) circle = tempCircle;` (`:210-211`) accepts a circle already proven not to contain all four points.
18. **Coplanar 3D hulls built as zero-thickness, doubled-face solids** (`ConvexHullAlgorithm.3D.cs:445-449`, both windings fanned from vertex 0). n<4 results (`:108-127`) have vertices but no faces/edges — `IsInside` (`ConvexHull3D.cs:112-118`) then returns `true` for *every* point.

Low

19. `RotatingCalipers2DMethod` iteration cap can end the sweep a step or two early (`MinimumEnclosure.cs:640`).
20. `ConvexHullAlgorithm.2D.cs:359`: collinear points kept only when the shared line isn't axis-parallel — inconsistent with "minimal" claims (`:40-41`).
21. `ConvexHullFace4D.GetNormal(true)` (`ConvexHull4D.cs:111-127`): `validNeighborCount` unused; returns `Vector4.Zero.Normalize()` (NaN) silently.
22. `Delaunay2D.CreateViaConvexHull` `reductionFactor` (`Delaunay2D.cs:44-45`) subtracts `avgX` from a value that is already a distance — formula wrong (harmless by scale invariance).

---

2. Redundancies

1. `MinimumCircleCylinderOld.cs` **is compiled, dead code — delete (High).** Lives in namespace `TVGL.Test` (`:19`); nothing references it; ships in the release assembly. Duplicates MaximumInnerCircle/MinimumBoundingCylinder nearly verbatim and retains a bug fixed in the new file (`:380-381` never updates `minCylinderVolume` → returns *last* cylinder, not smallest; compare `MinimumCircleCylinder.cs:399-404`). ~60 lines commented-out algorithms.
2. `GaussianSphere.cs` **is entirely dead (High).** ~730 lines, zero references, hardcoded 0.0001 tolerances, O(F^2) FindIndex scans — remove or archive.
3. **3D/4D quickhull duplication (Medium).** `AddVertexToProperFace` (3D:391-416 vs 4D:468-493), `GetExtremaOnAABB` (3D:596-636 vs 4D:671-717), main queue loop (3D:143-167 vs 4D:132-159) line-for-line; the shared `i += 2` bug is present in both — a generic core would land fixes once.
4. **KD-tree duplication (Medium).** `GenerateTree` copy-pasted between `KDTree.cs:336-407` and `KDTreeGenericWithAccompanying.cs:66-149`; ~20 Create overloads reduce to two generic ones (`:57, :105`).
5. **Dead/commented code in live files (Low):** `FindBestExtremaSubsetOLD` (`.4D.cs:516-543`); commented 5-vertex simplex (`:620-661`); ICP `U0`/`U1` computed never used (`IterativeClosestPoint.cs:45-46`), `norm()`/`AddTtoR`/`ZipMultiplyAdd` unreferenced; unused locals (`ConvexHullAlgorithm.2D.cs:316-317`); `GetDistanceToExtremeVertex<T>` unused generic param (`MinimumEnclosure.cs:409`); commented Presenter/CSV debug (`MinimumCircleCylinder.cs:83-108`).

---

3. Inefficiencies

1. **KD-tree search allocates two** `HyperRect`**s (four double[]) per node visited (High).** `KDTree.cs:430-434`; `HyperRect.cs:27-32`. Fix: mutate one bound in place, recurse, restore. ICP calls FindNearest per point per iteration (`IterativeClosestPoint.cs:65-67, 182`).
2. `BoundedPriorityList` **is a sorted List with Insert, not a heap (Medium).** `BoundedPriorityList.cs:110-136` O(k) memmoves per add (doc claims O(log n)); `numberToFind = -1` builds one sized to the whole tree with `allocate: false` (`KDTree.cs:319-321`). Use `PriorityQueue` or fixed-array max-heap.
3. **KD-tree construction garbage (Medium).** Per node: `points.Select(p => p[dim]).NthOrderStatistic(…)` copies into a fresh List (`StatisticCollection.cs:60`) + fresh left/right arrays (`KDTree.cs:342-350`). In-place Span quickselect over one working array. Also `Enumerable.Repeat(nullPoint, TreeSize).Cast<TPoint>().ToArray()` (`:279`) boxes every slot for struct points.

4. **GJK small fixed buffers (Medium).** `ConvexHullGJK.cs:44 new Vector3[4]`, `:107 new bool[3]` per call — stackalloc candidates. `support()` (`:546-565`) linearly scans all hull vertices per iteration; hill-climb over hull adjacency would be sub-linear.
5. **Missed parallelism over candidate directions (Medium).** `MinimumBoundingCylinder(vertices, directions)` runs 13 independent trials sequentially (`MinimumCircleCylinder.cs:397-405`); `FindMinimumBoundingBox`'s 13 ChanTan starts (`MinimumEnclosure.cs:198-204`). Natural Parallel.For min-reductions.
6. **`ConvexHull3D.LineIntersection` sorts all faces twice (Low).** `ConvexHull3D.cs:129, 139` — lazy OrderByDescending re-executed by Reverse(); a linear best/worst scan suffices.
7. **`Delaunay2D.Create` is O(n·m):** every insertion scans all triangles (`Delaunay2D.cs:185-192`); parallel circle List with O(m) RemoveAt (`:213-214`). Bowyer-Watson with locate-and-flood is the standard fix.
8. **LINQ in hot ICP loop.** `IterativeClosestPoint.cs:57-92` re-materializes half a dozen Lists per iteration; `CalHbCobig_Gabor` explodes matrices into 12+ List (`:238-269`) per iteration.
9. **2D hull scratch arrays:** `ConvexHullAlgorithm.2D.cs:294-301` allocates cvxVNum arrays each full input length (up to 4n tuples) — ArrayPool or growth-on-demand.

---

4. Other Concerns

1. **Library writes files to the user's Desktop.** `MinimumCircleCylinder.cs:104-107`: on iteration cap, writes `cvxpoints.csv` to Desktop in production code. Privacy/sandbox/server problem — behind a debug flag or remove.
2. **Console output from library code.** `ConvexHullAlgorithm.4D.cs:186`, `Delaunay2D.cs:107`, per-iteration `IterativeClosestPoint.cs:150` — route through Log.
3. **Non-deterministic randomness.** Unseeded `new Random()` in `.4D.cs:191` (jiggling → ConvexHull4D and Delaunay3D vary run-to-run), `Delaunay2D.cs:35`, `IterativeClosestPoint.cs:192, 203`. Accept a seed; reproducibility matters exactly when these degenerate paths fire.
4. **Thread safety.** `ConvexHull3D.Create(vertices, connectVerticesToCvxHullFaces/recordPartOfConvexHull)` mutates shared Vertex state (`.3D.cs:352-377`) — concurrent hulls over solids sharing vertices race. KD-tree immutable after construction, but nothing documents any of this.
5. **Tolerance zoo.** BaseTolerance 1e-9, DefaultEqualityTolerance 1e-12, OBBTolerance 1e-5 (`Constants.cs:83, 99, 170`) + ad-hoc 0.0001/0.00001/1.0001/sqrt(BaseTolerance) literals, mostly absolute, not model-scaled. The 4D AABB-diagonal-derived tolerance (`.4D.cs:62`) deserves to be the norm.
6. **Readability.** `ConvexHullGJK.S3D` ~370 lines of transliterated C; ICP interleaves MATLAB comments; 2D `Create` ~250 lines mixing three phases; 4D "stack" is actually a Queue (BFS) while the comment says depth-first (`.4D.cs:250-253`); self-review comments left in (`.3D.cs:520-521`).

---

5. API Usability Observations

1. **Two competing idioms for "get me an enclosure":** static try-pattern factories (`ConvexHull3D.Create(pts, out hull, out indices) : bool`) vs extension methods returning directly and throwing (`points.MinimumCircle()`, `.MinimumSphere()`, `.FindMinimumBoundingBox()`). Enclosure functionality also split across three entry-point types.
2. **Output-type inconsistency.** ConvexHull3D is a full Solid; ConvexHull4D a bare class of arrays; ConvexHull2D returns List + parallel out List of indices; BoundingBox vs BoundingBox both exist (`MinimumEnclosure.cs:794-818`); Delaunay return raw arrays.
3. **Naming traps.** `ConvexHull4D.Faces` are `Edge4D` objects that are actually *triangles*; true edges called `VertexPairs` (`ConvexHull4D.cs:25-30`). Public lowercase `tolerance` (`ConvexHull3D.cs:27`, doc says "The volume of the Convex Hull."); `peakVertex`/`peakDistance` public-lowercase. `CreateConvexHullMinimal` accepts a `tolerance` it never uses (`.2D.cs:48-74`).
4. **Silent nulls and misleading flags.** `MinimumBoundingCylinder(IList, direction)` returns null for null/zero direction (`:418-419`) while the multi-direction overload then NREs; `Delaunay3D reuseInputVertices` doesn't; `out vertexIndices` broken — the advertised way to map hull points back to inputs doesn't work.
5. **KD-tree surface.** Twenty near-identical Create overloads, yet dimension — the one thing that matters — is hardwired to 3 and wrong for 2D. `FindNearest(target, numberToFind = -1)` = "all points, sorted" is a surprising default.

Top-priority fix list

1. KD-tree Dimensions = 3 for 2D points (wrong nearest neighbors).
2. GetDistanceToExtremePoints bottom/top swap.
3. Stale-rectangle side points in rotating calipers.

4. vertexIndices out-param broken in 3D/4D hull Create.
5. (long)(n * n) overflow + one-sided jiggle in 4D.
6. Delete MinimumCircleCylinderOld.cs and GaussianSphere.cs (dead, compiled, one carries a known bug).
7. Absolute iteration caps for MinimumSphere/MinimumGaussSpherePlane; remove Desktop CSV write and Console.WriteLines.

Appendix E — Implicits and Primitives

Code Review: `Implicits and Primitives` (TVGL)

Scope: all 29 files under `Implicits and Primitives\`. All findings verified by reading cited code.

---

1. Potential Errors

High

- **BoundingBox.cs:193 — `Bounds` computed from unit vectors, not scaled vectors.** `GetBoxBounds(Directions, TranslationFromOrigin)` passes normalized `Directions`; `GetBoxBounds` (:198) expects dimension-scaled `Vectors`. Every box with dimensions ≠ 1 gets a wrong AABB; also breaks `Intersects` (:649) for axis-aligned pairs.
- **BoundingBox.cs:549–552 — `Copy()` loses the box dimensions.** Constructor falls back to unit lengths → copy is a 1×1×1 box. (`BoundingBox<T>.Copy` at :98 passes Dimensions correctly.)
- **BoundingBox.cs:563–576 — `MoveFaceOutward` destroys dimensions** (builds from unit Directions; result dimensions `(1+distance, 1, 1)`).
- **BoundingBox.cs:364 — `Center` ignores dimensions** (uses unit directions; must use Vectors).
- **BoundingBox.cs:598 — `LineIntersection` upper point wrong for oriented boxes** (`TranslationFromOrigin + Dimensions` adds dimensions as global-frame vector).
- **BoundingBox.cs:255–264 — `TransformFromUnitBox` missing the scale** — byte-for-byte same as `TransformFromOrigin`; round-trip with `TransformToUnitBox` (:272) ≠ identity.
- **StraightLine3D.cs:123 — wrong direction in 2-point fit.** `(dir - anchor).Normalize()` where dir = p2 − anchor → direction = p2 − 2·anchor. Should be `dir.Normalize()`.
- **StraightLine3D.cs:134–139 — singular-matrix fallback always yields UnitZ** (missing `else`).
- **Cone.cs:208–210 — `TransformFrom2DTo3D` adds the Y-term twice, omits the X-term.** Every 2D→3D mapping on a cone is wrong.
- **GeneralQuadric.cs:252 — quadric transform uses the matrix instead of its inverse.** $M \cdot Q \cdot M^T$ instead of $M^{-1}\ Q\ M^{-T}$ — surface transformed by the *inverse* of the requested transform (correct only for origin rotations).
- **GeneralQuadric.cs:882 — sphere→quadric W term typo.** `Center.Z + Center.Z` should be `Center.Z * Center.Z` — every converted sphere has wrong constant term.
- **GeneralQuadric.cs:1020–1021 — `SetQuadricType` builds the A-matrix with `YSqdCoeff / 2` on the diagonal** (should be YSqdCoeff) → quadric classification unreliable whenever YSqdCoeff ≠ 0.
- **GeneralQuadric.cs:739–742 — least-squares fit error computed incorrectly.** Adds `W` once instead of N·W; `errSqd *= errSqd / numVerts` gives $(\Sigma Q)^2/N$ not $\Sigma(Q^2)/N$.
- **Capsule.cs:264 — double subtraction in cone-section distance.** `t = (dxAlong - conePlaneDistance1) / coneLength` but dxAlong already relative to coneAnchor1 (:258). Should be `t = dxAlong / coneLength`. Wrong for any capsule not near the origin.
- **CappedCylinder.cs:50–64 — `DistanceToPoint` wrong beyond the caps** (returns distance to cap rim circle instead of axial distance to cap face); `IsPositive` sign flip (:68) applied only in-body branch.
- **Prismatic.cs:180–190 — constructor uses `Vertices` before they exist** (`GetDistanceToExtremeVertex(Vertices, …)` at :184 before `SetFacesAndVertices(faces)` at :189 → null; compare Cylinder.cs:261–272 correct ordering).
- **Torus.cs:147 — inverse transform built by negating a matrix.** `backTransform * -translate` — operator `-` negates every element, not the inverse of a translation. `TransformFromYPlane` (typo'd name, :132) returns garbage.
- **GeneralConicSection.cs:306 — circle center typo:** `new Vector2(x, x)` should be `(x, y)`.
- **PrimitiveSurfaceExtensions.cs:1031–1032 — `TessellateHollowCylinder` axis offsets inverted sign** (compare correct `Tessellate(Cylinder)` :951). Tube mirrored about anchor.
- **PrimitiveSurfaceExtensions.cs:1073 — `TessellateHollowCylinder` omits `innerFaces` from the solid** → open inner wall; also `btmPlane` (:1076) uses +Axis where :989 correctly uses −axis.

- **SphericalZBuffer.cs:106 — base `XLength` never set** (local shadows inherited property) → inherited wrap logic (CylindricalZBuffer.cs:185–208) adds XLength (=0) — seam wrapping broken for spherical buffers. MaxX/MaxY/YLength also unset.
- **CylindricalZBuffer.cs:398 / SphericalZBuffer.cs:148 — `Get3DPoint` double-counts the radius** (`radius = baseRadius + flatPoint.Z` where heights already absolute).
- **Plane.cs:490–496 — `DotCoordinate` sign wrong for TVGL's convention.** Computes `N·p + DistanceToOrigin` vs `DistanceToPoint` (:579) `N·p − DistanceToOrigin` — System.Numerics port where D = −distance wasn't sign-flipped.

Medium

- **Sphere.cs:282 — `PointMembership` gradient wrong** (`distance * v / vLength`; gradient of |v|−R is v/|v|; three-out-param overload :298 correct).
- **Sphere.cs:358–364 — tangent line yields duplicate intersections** (missing `yield break`; 3 results possible). Cylinder.cs:376–385 identical flaw.
- **Cylinder.cs:370–386 — line parallel to axis divides by zero** → two garbage intersections instead of none.
- **CappedCylinder.cs:123–158 — `LineIntersection` misses axis-parallel lines through the caps**; cap hits never radius-checked when only one wall intersection.
- **Torus.cs:434–452 — `LineIntersection` mixes normalized and unnormalized directions** (quartic solved with normalized, results reconstructed with caller's raw direction). `lineT` conventions inconsistent across surfaces.
- **Torus.cs:122–150, 155–174 — stale cached transforms** — never invalidated in `Transform()` or Center/Axis setters.
- **Torus.cs:430 — `GetSegmentsWithRadius(target, …)` compares against `MinorRadius` instead of `target`** (latent); double enumeration via Count() (:415/419).
- **Plane.cs:446–481 — `Transform(Quaternion, bool)` never transforms faces/vertices** (calls base with Identity); cached XY transforms not invalidated.
- **PrimitiveSurface.cs:173 — dead NaN guard** (`if (double.IsNaN(d)) ;`) — NaN poisons mse.
- **PrimitiveSurface.cs:953–989 — `Copy(bool)` leaves stale cached references** (`_largestFace`, `_adjacentSurfaces` point at original solid; `ResetFaceDependentValues` doesn't clear them).
- **SurfaceGroup.cs:192–211 — closest-surface selection uses signed distance** (deeply-inside negative always wins). Use |d|.
- **SurfaceGroup.cs:71–117 — Faces/Vertices/KeyString caches never invalidated** after AddPrimitiveSurface/Combine.
- **UnknownRegion.cs:198–204 — inverted face pick in `GetNormalAtPoint` edge branch** (sentinel +∞ guarantees the excluded face is chosen).
- **Circle.cs:249–260 — `IntersectWithCircle` returns NaN for contained circles** (only `> R1+R2` rejected; `< |R1−R2|` → sqrt(negative), returns true with NaN points).
- **GeneralConicSection.cs:162–171 — inconsistent discriminant conventions** ($B^2 \approx A{\cdot}C$ vs $A{\cdot}C - B^2$ vs correct $4AC - B^2$ at :433); `B = Math.Sqrt(A*C)` can be NaN and drops sign (:164).
- **PrimitiveSurfaceExtensions.cs:796 — `Voxelize` compares a line parameter to a coordinate** (`lineT > result.XMax`; should be XMax − XMin); minJ/minK not clamped ≥ 0 (:781–784); `GetPrimitiveAndLineIntersections` (:1245) anchors ray at x=0 in faces branch but x=XMin in primitive branch — two frames, filter wrong for one.
- **ZBuffer.cs:113–124 — `PrimitiveZmins` path nearly unreachable and NRE-prone.**
- **CylindricalZBuffer.cs:168–174 — faces with 1 or 3 wrapping edges silently dropped** (empty-statement branches; comments admit cases occur).
- **BoundingBox.cs:649–668 — OBB `Intersects` incomplete** (corner containment both ways misses SAT edge-edge cases).
- **BorderLoop.cs:118–127 — `Curvature` setter overridden by lazy getter** (doesn't set `_curvatureIsSet`; silently breaks `Copy` :261).
- **BorderLoop.cs:343–355 — `UpdateIsClosed` throws on empty loop** (indexes `SegmentDirections[^1]`).
- **ImplicitSolid.cs:285–292 — mixed field conventions in CSG evaluation** (QuadricValue algebraic vs metric signed distance; some surfaces return unsigned distances in regions) — min/max CSG not commensurate; breaks `PointIsInside`.

Low

- Cone.cs:313–336 — `SetPrimitiveLimits` lacks the NaN guards Cylinder.cs:328–333 applies; Torus.cs:399–410 likewise.
- Cone.cs:254–257 — asymmetric wrap correction (subtracts 2·halfRepeatAngle one way, adds halfRepeatAngle the other).
- Cone.cs:307–308 — `CalculateIsPositive` mutates `Axis` as a side effect of a lazy property read.
- Helix.cs:82 — `DeterminePitch` divides by `line.Direction.X`, no zero check.
- GeneralQuadric.cs:362 — `new Random()` inside retry loop; offset biased to positive octant.
- GeneralQuadric.cs:842 — `DefineAsCone` NotImplementedException for axis-aligned-Y; `DefineAsCylinder` (:808) divides by axis.Z; sqrt calls can yield NaN (780, 789, 811).

- Plane.cs:519–525 — `SameLocation` exact `==` on doubles; doesn't recognize negated normal/distance.
- Capsule.cs:229–246 — degenerate `Anchor1 == Anchor2` unhandled (NaN).
- StraightLine2D.cs:115 — exact `yCoeff == 0` branch; 1/xCoeff Infinity if also ~0.
- Sphere/Cone/Torus `faceXDir` lazily defaults — same 3D point maps to different 2D coords depending on call history.
- **Serialization:** GeneralQuadric.cs:21 uses System.Text.Json `[JsonIgnore]` while library serializes with Newtonsoft → attributes ignored, StationaryPoint's solve may run during serialization. `PrimitiveSurface.FaceIndices` (:274) refreshes only when shrunk → stale after face loss. `BorderSegment.CircleCenter` (:152) public field. `BorderSegment.Copy` (:293) triggers SetCurve on source; copy's `_curve` discarded.

---

2. Redundancies

- **CappedCylinder.cs:98–115 duplicates Cylinder.cs:320–346** (`SetPrimitiveLimits` finite branch verbatim minus NaN guards). High-value cleanup.
- **CylindricalZBuffer.cs:210–327 duplicates ZBuffer.cs:242–396** — entire ~120-line triangle spine-walk rasterizer copy-pasted (only `xIndex % XCount` differs). A `WrapX(int)` hook collapses them. Same for wrap preamble at CylindricalZBuffer.cs:183–209 vs 334–366.
- **ZBuffer.cs:144–169 vs 175–185** — `UpdateZBufferWithSurface` re-implements `UpdateZBufferWithFace` inline.
- **Prismatic.cs:408–416 == UnknownRegion.cs:225–233** — identical face-ray LineIntersection loops (also PrimitiveSurfaceExtensions.cs:1243–1249).
- **Prismatic.cs:304–374** — DistanceToPoint/ClosestPointOnSurfaceToPoint/GetNormalAtPoint each repeat the same closest-segment scan.
- **Sphere/Cylinder/Cone/Torus/Prismatic** `Transform` all repeat the "transform two perpendicular radius vectors and RMS-average lengths" idiom — extractable helper.
- **Dead code:** Prismatic.cs:379–399 (unreachable after return); Capsule.cs:289–297 (unreachable after throw); PrimitiveSurface.cs:509–544 (~35 commented lines); Cylinder.cs:389–404 (commented AsTessellatedSolid); ImplicitSolid.cs:463–465 (duplicate if); Grid.cs:341–342 (debug no-op); ZBuffer.cs:171–173 orphaned doc.
- **Unused usings:** GeneralQuadric.cs:14/20, Capsule.cs:17, BoundingRectangle.cs:17, Grid.cs:14.
- **SurfaceGroup.cs:171–175 —** `new SetColor` shadow-hides an equivalent base method.
- **Overlapping ZBuffer implementations:** ZBuffer/CylindricalBuffer/SphericalBuffer carry own `Initialize` via `new`-hiding (CylindricalZBuffer.cs:117, SphericalZBuffer.cs:101) — redundancy + correctness trap (base-typed call runs wrong one; see XLength bug).

---

3. Inefficiencies

- **Prismatic.cs:281–298 — Dijkstra node stores full copied path list per node** (O(V^2)); store parent ref. Line 267 uses `LengthSquared()` as additive edge weight — doesn't minimize geometric length.
- **GeneralQuadric.cs:380–390 —** `GetNearbyPointOnQuadric` **re-enumerates** `LineIntersection` via `GetEnumerator().MoveNext()` in loop condition and again at :389/390; enumerators never disposed.
- **UnknownRegion.cs:151–188 —** `GetClosest` **full O(faces+edges+vertices) scan per query** with per-call Concat allocation; three public methods each redo it.
- **ZBuffer rasterization single-threaded:** ZBuffer.cs:116–117, CylindricalZBuffer.cs:99–100; `Voxelize` has Parallel.For scaffolding commented out (:787, :802 — idempotent writes, safe). `Grid.ConvertGridToPolygons` triple-N scan (Grid.cs:423–427) also parallel-friendly.
- **Multiple enumeration of IEnumerable params:** StraightLine3D.cs:86/119/177; Circle.cs:105/182; StraightLine2D.cs:79/123; Plane.cs:116–128 (+`faces.Count()` in SetFacesAndVertices, PrimitiveSurface.cs:75); Sphere.cs:371–375; `SphereIsTooFlat` (:381) O(n^2) when O(n) suffices.
- **Cylinder.cs:355 — instance** `LineIntersection` **calls** `.ToList()` on 0–2 element iterator per ray test (hot path in Voxelize).
- **Per-call allocations:** `BoundingBox.LineIntersection` (:599–601) 6-element array + List per call; `SortedDimensionsShortToLong`/`SortedDirections…` (BoundingBox.cs:469–539) fresh arrays per property read; `ShortestDimension`/`LongestDimension` allocate an array to read one element.
- **Span opportunities:** `GetVerticesFromModifyEdges` (PrimitiveSurfaceExtensions.cs:663), `GetCircleTessellation` pooled/stack storage; Grid deltas jagged int[][] (Grid.cs:530) → flat ReadOnlySpan.
- **Virtual dispatch in tight loops:** `GetIndicesCoveredByFace`/`PixelIsInside` virtual per pixel; seal leaf classes (ZBuffer doc says "cannot be inherited" but class not sealed, ZBuffer.cs:21–23); `SurfaceGroup.DistanceToPoint` LINQ Min over polymorphic calls.

---

4. Other Concerns

- **Inheritance design.** `SurfaceGroup : PrimitiveSurface` implements ~10 abstract members as `throw NotImplementedException` (177–205, 223) — composition fits better. `CappedCylinder : Cylinder` means capped is-a infinite cylinder to type checks (e.g. `GeneralQuadric.FromPrimitiveSurface` :883 converts capped → infinite quadric cylinder). ZBuffer family's `static new Run(...)` that throws (CylindricalZBuffer.cs:37–39, SphericalZBuffer.cs:36–38) and `new`-hidden Initialize are the weakest parts.
- **Mutable state after construction.** Nearly all primitives expose `{ get; set; }` geometry with lazy caches that don't watch them: `Cylinder.Axis` set doesn't clear faceXDir/faceYDir (Cylinder.cs:50–76); `Plane.Normal` set doesn't clear `_asTransformToXYPlane` (Plane.cs:58–94); Torus above. GeneralQuadric uses init coefficients but exposes mutable public fields a,b,c,d,e (:144–148) and settable Type. `Sphere/Cone/Torus.TransformFrom3DTo2D(IEnumerable…)` *mutate* faceXDir/faceYDir/faceZDir as a side effect of a query (Sphere.cs:227–238) — order-dependent, thread-unsafe.
- **Tolerance handling inconsistent.** Mix of Constants.BaseTolerance, bare literals (1E-3 RemoveSmallCoefficents, 1E-12 SetQuadricType, 1E-4/1E20 SQP loop GeneralQuadric.cs:419/427, 0.02 rad Circle.cs:173, 1.67π PrimitiveSurfaceExtensions.cs:108, dotAligned 0.999 BoundingBox.cs:136), scale-dependent absolute checks. `PixelIsInside` (ZBuffer.cs:403/419) mixes strict with IsLessThanNonNegligible on one barycentric only.
- **XML docs:** boilerplate ("Points the membership." on every DistanceToPoint), `<c>true</c> if XXXX` placeholders, wrong summaries (Cylinder.Tessellate documented "for a cone" PrimitiveSurfaceExtensions.cs:921–928; Torus.KeyString doubled "|"; "Primsatic|" typo Prismatic.cs:144), param lists not matching signatures (Sphere.cs:338–349). `Cone.PracticalMinAperture/MaxAperture` (Cone.cs:26–27) public consts unused.
- **`BoundingRectangle.CornerPoints` throws** on 1% area mismatch (BoundingRectangle.cs:146) — assertion as control flow in a read-only accessor.

---

5. API Usability Observations

- **Fitting entry points scattered and inconsistent:** `Plane.FitToVertices`/`DefineNormalAndDistanceFromVertices`, `Sphere.FitToVertices`/`DefineSphereFromVertices`/`CreateFrom{2,3,4}Points`, `GeneralQuadric.DefineFromPoints`, `Circle.CreateFromPoints`, `StraightLine3D.CreateFromPoints`. Return styles differ (nullable vs bool+out vs always-succeed); error semantics differ. A uniform `TryFit(points, out surface, out error)` per primitive would help.
- **Property naming exceptions:** Cone `Aperture` is a *slope* not an angle (doc :63–68 must explain; rename `HalfAngle`/`ApertureSlope`); Sphere/Torus `Center` vs Cylinder `Anchor` vs Capsule `Anchor1/2`; Prismatic `BoundingRadius`. `Cylinder.MinDistanceAlongAxis` is offset from *origin* along axis, not from Anchor — undocumented; TessellateHollowCylinder got it wrong.
- **`Circle(Vector2 center, double radiusSquared)` is a foot-gun** (Circle.cs:60) — passing a radius compiles fine; no FromRadius factory.
- **`IsPositive`-dependent signed distances surprising:** sign flips with IsPositive on Plane/Sphere/Cylinder/Cone/Torus/Capsule but not Prismatic, partially CappedCylinder, via QuadricValue on GeneralQuadric. Documenting/enforcing one convention on the abstract `DistanceToPoint` is the single highest-leverage doc fix.
- **65 KB `PrimitiveSurfaceExtensions` mixes ≥5 concerns:** border/axis queries, generic property access (GetAxis/GetAnchor/GetRadius), mesh surgery (TrimTessellationToPositiveVertices ~250 lines), tessellation factories (Tessellate ×9), voxelization. Split into PrimitiveBorderExtensions/PrimitiveTessellation/PrimitiveTrimming. GetAxis/GetAnchor/GetRadius (176–230) are the de-facto polymorphic API — better as virtual members on PrimitiveSurface so new primitives can't be forgotten (Capsule was bolted on; GetRadius mixes if/else-if styles :216–219). `GetAxis` on GeneralQuadric returns unordered eigen axis — unspecified.
- **`ICurve` static-abstract `CreateFromPoints` constrained to `IVector2D`** (ICurve.cs:43) forces 3D implementers into runtime downcasts (StraightLine3D.cs:88); `Helix.CreateFromPoints` just throws. Split ICurve2D/ICurve3D.
- **BoundingBox ergonomics:** three near-identical transform property names where two are currently identical (bug); `MoveFaceOutward(int, bool, …)` hard to call correctly (and broken); the CartesianDirections overload (:583) is the usable one.

Appendix F — InputOutput Operations

Code Review: TVGL `InputOutput Operations` (+ SolidAssembly.cs)

Abbrev: `IO/` = `InputOutput Operations\`. Every finding verified by reading the cited code; XmlSerializer collection-initialization behavior verified empirically on net10.0 (several "null collection" suspicions were checked and dropped).

---

1. Potential Errors

High

- **Culture-sensitive double parsing breaks read AND write on comma-decimal locales.**
  - `IO/IOFunctions.cs:738, 762` — `Double.TryParse` with no `CultureInfo.InvariantCulture` in `TryParseDoubleArray` — the parse path for **ASCII STL** (`STLFileData.cs:183,191`) and **OFF** (`OFFFileData.cs:164,182,205`). On fr-FR/de-DE, `"1.5"` fails → IOException or false.
  - `IO/IOFunctions.cs:1121-1249` — all three `ReadNumberAsInt/Float/Double(string, Type)` overloads culture-sensitive — the **ASCII PLY** value parsers (`PLYFileData.cs:556,572,621,624`).
  - `IO/3mf.classes.cs:110` — 3MF `transform` attributes → NaN on comma locales.
  - **Writers equally broken:** ASCII STL `STLFileData.cs:368-372`, OFF `OFFFileData.cs:288-306`, ASCII PLY `PLYFileData.cs:812,821` use current-culture ToString → files containing `1,5` no tool can re-read. Only OBJ (`OBJFileData.cs:326-331,372,409`) and CSV/SVG/DXF polygon readers (`IOFunctions.cs:503,555,636`) are correct.
- `File.OpenWrite` **never truncates — saving over a longer existing file leaves trailing garbage.** `IOFunctions.cs:1270, 1351, 1435`. Should be `File.Create`. (Polygon saver deletes first, `:1516`.)
- **Binary STL color channels swapped between reader and writer.** Reader: blue bits 0–4, red bits 10–14 (`STLFileData.cs:294-313`); writer packs red 0–4, blue 10–14 (`:446-452`). TVGL-written colored binary STL re-opened by TVGL has R/B exchanged.
- **Binary STL writer can emit a header shorter than 80 bytes.** `STLFileData.cs:406-408` truncates >80 but never pads shorter → 4-byte face count lands inside header → unreadable everywhere. Truncation by char count then UTF-8 re-encode — non-ASCII comments can exceed 80 bytes.
- **OBJ:** `w` **coordinate parsed from wrong index.** `OBJFileData.cs:331` — `values[2]` should be `values[3]`. 4-component vertices divide x,y,z by **z**.
- **OBJ: middle** `g` **groups silently dropped.** `OBJFileData.cs:259-277` — new `objSolid` in the else branch never added to `objData`; only the final one rescued at `:313-314`. For `g A / g B / g C`, solid B's faces lost entirely.
- **OFF files routed to the PLY parser in two of three Open dispatches.** `IOFunctions.cs:313-316` and `:413-416` send `FileType.OFF` to `PLYFileData.OpenSolid` → returns null → array overload returns `true` with `new[]{ null }`. Only single-solid overload (`:148-149`) uses OFFFileData.
- **Binary PLY header offset wrong for CRLF and BOM.** `PLYFileData.cs:143-144, 220-221` count `line.Length + 1` assuming LF-only, no BOM; seek at `:151` lands mid-header/mid-data for Windows-authored binary PLY. (TVGL's own writer strips CRs at `:856-862` for this reason.)
- **Binary PLY: non-color extra vertex properties desynchronize the stream.** `PLYFileData.cs:755-786` — when property is not xyz and `vertexColorDescriptor == null`: NRE on null `vertexColors`, and the property's bytes are **never read** → PLY with nx/ny/nz normals (extremely common) crashes or reads misaligned garbage. Color path also assumes every non-xyz property is a color (`:761`).
- `SaveToString` **produces mojibake for every format.** `IOFunctions.cs:1330-1336, 1412-1418, 1450-1456` decode with `Encoding.Unicode` (UTF-16) but writers emit UTF-8; also mismatches `OpenFromString` (UTF-8, `:215`) — advertised round-trip cannot work.
- **All writers silently drop user comments/metadata: inverted Where predicate.** `.Where(string.IsNullOrWhiteSpace)` keeps *only blank* comments in six places: `STLFileData.cs:349, 404`; `OFFFileData.cs:286`; `PLYFileData.cs:927`; `3MFFileData.cs:347`; `AMFFileData.cs:251`. Metadata reconstruction blocks (`3MFFileData.cs:348-361`, `AMFFileData.cs:253-265`) are dead code in practice.

Medium

- `ReadExpectedLine` **rewind fundamentally broken with buffered StreamReader.** `IOFunctions.cs:777-785` — saved BaseStream.Position already past read data; reset doesn't discard reader buffer; throws on non-seekable streams. Used by ASCII

STL (`STLFileData.cs:187,204`).

- **3MF `OpenModelFile` NREs on malformed input; try/catch commented out.** `3MFFileData.cs:134–135, 169–175`; `md.type` null → NRE at `:154`.
- **3MF `.rels` output invalid.** `3MFFileData.cs:421–423` serializes only `rels[0]` as bare `<Relationship>` root — no `<Relationships>` container; thumbnail relationship never written yet empty `Metadata/thumbnail.png` entry created (`:290`).
- **3MF comments lost even without the filter bug.** `3MFFileData.cs:89–97` — Comments getter builds a new list each call; `AddRange` at `:376` mutates a temporary.
- **Binary STL: strict length check rejects legitimate files and defeats the multi-solid loop.** `STLFileData.cs:243–245`; re-evaluated with same `length - 84` on second do-while iteration (`:241–257`). Failure falls into `TryReadAscii` which **always returns true** (`:172`) → silent empty result.
- **ASCII STL: solid name assignment bug.** `STLFileData.cs:157–160` — missing else: default name unconditionally overwritten; unnamed solids get "". Successor solids (`:168`) lose FileName/Units.
- **PLY vertex-color averaging divides by 4 but sums 3 vertices** (`PLYFileData.cs:174–182`, 25% darkened). Also both readers push one Color per color *component* not per vertex (`:646, :783` inside the per-property loop) → vertex colors scrambled.
- **Binary PLY with an `edge` element always throws.** `PLYFileData.cs:490–492` → `ReadEdges(BinaryReader) = throw NotImplementedException` (`:662–664`); ASCII version skips gracefully (`:534–542`).
- **`BackoutToFolder`/`BackoutToFile` NRE at filesystem root.** `IOFunctions.cs:243–264` — loop condition dereferences `dir.FullName` before the null guard inside.
- **OFF writer emits corrupt face lines when a face has no color.** `OFFFileData.cs:270, 304` — missing leading space: `"3 0 1 20.5 0.5 0.5"`. Writes 4 color components (`:301–302`) while reader accepts exactly 3 (`:223`) — colors never round-trip; `solid.SolidColor` NRE if null (`:270`).
- **OBJ multi-solid early return.** `OBJFileData.cs:132` — `if (!tsBuildOptions.PredefineAllEdges) return new[] { ts };` inside the per-solid loop discards all solids after the first.
- **Zip/stream disposal gaps in 3MF read.** `3MFFileData.cs:115` ZipArchive never disposed; `:118` deflate stream never disposed (XmlReader CloseInput=false default).
- **`SolidAssembly.StreamRead` re-adds solids to an already-deserialized RootAssembly.** `SolidAssembly.cs:261–264` — duplicates part references with Identity transforms; relies on dictionary insertion order.
- **`SaveToString(Solid)` with the default argument always throws.** `IOFunctions.cs:1330` defaults to `FileType.unspecified` → NotSupportedException (`:1319–1322`).

Low

- Host little-endianness assumed throughout `ReadNumberAs*` (`IOFunctions.cs:872–1064`) and binary writers; `BinaryPrimitives` would be explicit and faster.
- Non-seekable streams break format fallback (`STLFileData.cs:106`, `OFFFileData.cs:122`, `PLYFileData.cs:151`).
- `OFFFileData.cs:159,177,199` — `line.Remove(0, 1)` result discarded (no-op).
- `TryParseNumberTypeFromString`: `uint64` captured by the `long` branch (`IOFunctions.cs:1080–1083`, ulong branch unreachable); `int8` maps to `byte` (`:1101`).
- `InferUnitsFromComments` regex lowercase-only (`:813`) — "MM"/"Inches" → mangled tokens.
- PLY: NRE on empty file (`:140`), missing end_header (`:220`); `element uniform_color` written with no count (`:947`) — non-standard.
- 3MF `requiredextensions` (`3MFFileData.cs:103`) and AMF `version` (`AMFFileData.cs:88`) lack `[XmlAttribute]`.
- AMF supports only one `volume` per mesh (`amf.classes.cs:76`) — multi-volume meshes lose all but first.
- `CurrentCultureIgnoreCase` for token comparisons everywhere — should be OrdinalIgnoreCase (tr-TR I/i).

---

2. Redundancies

- **High — the Open dispatch switch is written three times** with drifting behavior (source of the OFF→PLY bug): `IOFunctions.cs:130–159, 295–332, 390–428`. One private dispatcher would collapse these.
- **High — `ReadNumberAsInt/Float/Double` in six near-identical copies** (3× BinaryReader `:872–1064`, 3× string `:1117–1250`), each an if-ladder over 9 Types. Generic helper/TypeCode switch removes ~350 lines.
- **Medium — PLY color-property recognition copy-pasted 3×** (`PLYFileData.cs:280–312, 341–374, 381–417`); whole ReadMesh/ReadVertices/ReadFaces/ReadUniformColor family duplicated for StreamReader vs BinaryReader (`:436–499, 506–792`).
- **Medium — `"type ==> value"` metadata split duplicated** in 3MF (`:347–361`) and AMF (`:251–265`), same `//todo` in both.

- **Medium — `TVGLFileData.OpenTVGLz` overloads byte-for-byte duplicates** (`TVGLFileData.cs:46-67` vs `:73-93`); three `OpenFromString` overloads too (`IOFunctions.cs:213-240, 349-360, 444-455`).
- **Dead code:** commented serializer experiments (`TVGLFileData.cs:34-40, 215-239`); `OFFFileData.TryReadBinary` = `return false; throw ...` (`:246-251`); unused `ConvertDoubleArrayToString/...` (`IOFunctions.cs:1550-1568`); `SolidAssembly` serialization hooks gated by always-false flag (`:270-297`); OBJ `FaceToNormalIndices` populated never consumed (`OBJFileData.cs:305`); PLY dead null check (`:408`); `STLFileData.Normals` collected never used (`:206,321`).
- **Legacy cruft:** dual 2013/01 vs 2015/02 3MF material models (`3mf.classes.cs:481-645`); `#if help` toggles across 3mf/amf classes.

---

3. Inefficiencies

- **High — ASCII STL keeps the entire file in a List and regex-scans all of it.** `STLFileData.cs:146-150` adds every line (7/facet) to `comments`; `InferUnitsFromComments` (`IOFunctions.cs:809-818`) regexes every one at each endsolid. 1M-facet STL ≈ 7M regex invocations + O(file) memory. Only `#` comment lines should be collected.
- **High — ASCII STL facet parsing: two Regex objects per line** (`STLFileData.cs:50,55` at `:183,191`). `ReadOnlySpan<char>` slicing + `double.TryParse(span, InvariantCulture)` = order-of-magnitude win, fixes culture bug simultaneously.
- **High — binary parsing allocates a byte[] per scalar + LINQ `Reverse().ToArray()`.** `IOFunctions.cs:872-1064`; a binary STL face = 13 allocations (`STLFileData.cs:267-288`). Read each 50-byte facet into stackalloc/reused buffer + `BinaryPrimitives.ReadSingleLittleEndian`. Same for binary PLY (`PLYFileData.cs:705-792`). Also `:288-313` decodes attribute word via `Convert.ToString(value, 2).PadLeft(16,'0').ToCharArray()` — bit ops suffice.
- **Medium — per-line string.Split across OBJ/PLY/OFF.** OBJ re-splits every vertex token twice (`OBJFileData.cs:303, 348, 370`); PLY per vertex/face line (`:555,615`); OFF `Split().ToList()` + RemoveAll per line (`IOFunctions.cs:733-734`). `MemoryExtensions.Split` over spans removes per-line arrays.
- **Medium — dispatch on System.Type objects per value** (`List<Type>` + if-ladder of `type == typeof(...)` per scalar); resolve an enum once in the header.
- **Medium — no parallelism for independent meshes.** 3MF objects (`3MFFileData.cs:161-165`), multi-solid ASCII STL (`STLFileData.cs:117-123`), OBJ sub-solids (`OBJFileData.cs:125-138`) build TessellatedSolids sequentially; data-independent, trivially Parallel.ForEach-able.
- **Medium — PLY binary save header hack** (`:853-862` MemoryStream→ToArray→ToList→RemoveAll(13)→ToArray) — `StreamWriter.NewLine = "\n"` does it with zero copies.
- **Low —** writers concatenate strings per line; `SaveBinary` STL allocates via BitConverter per float (`STLFileData.cs:431-454`); `GetMostSignificantSolid` re-enumerates its lazy Where chain five times (`IOFunctions.cs:181-204`); no BufferedStream for BinaryReader paths.

---

4. Other Concerns

- **Error reporting inconsistent and mostly invisible (Medium).** Failures logged as Information with magic verbosity ints, occasionally Log.Error, sometimes raw Debug.WriteLine. Array-overload catch (`IOFunctions.cs:334-339`) swallows everything incl. the NREs above; single-solid `Open<T>` has no catch — two contradictory contracts for the same files.
- **Extension dispatch (Medium).** Unknown extension on save → NotImplementedException (`:699`); on open → log-and-return. SHELL/DXF/DWG/SVG/CSV only partially wired.
- **XML (Medium/Low).** 3MF read via XmlReader.Create is XXE-safe; AMF via `XmlSerializer.Deserialize(TextReader)` (`AMFFileData.cs:108-110`) doesn't prohibit DTD processing — wrap with `DtdProcessing.Prohibit` to close billion-laughs DoS.
- **Newtonsoft.Json (Low/Medium).** `Save(Polygon)` uses `TypeNameHandling.Auto` (`IOFunctions.cs:1505`) while `OpenPolygonFromJson` uses a default serializer (`:619-625`) — polymorphic content won't round-trip; never enable TypeNameHandling on read without a binder (gadget risk). Hand-rolled streaming via WriteRawValue (`SolidAssembly.cs:195-197`) is fragile.
- **Zip (Low).** No zip-slip (nothing extracted to disk); no decompression-bomb guard in 3MF/TVGLz; 3MF entry discovery by `.EndsWith(".model")` ignores OPC `_rels` indirection — silently loads nothing for unconventional packages.
- **`SolidAssembly.StreamWrite` silently drops non-tessellated solids (Medium).** Only TessellatedSolids written (`SolidAssembly.cs:204`); CrossSectionSolid/VoxelizedSolid vanish on save with no warning. `TVGLFileData.SaveToTVGL(Stream, Solid)` hard-casts (`:195`); bare catch (`:208`) converts InvalidCastException into silent false.
- **No async surface (Low).** Everything sync I/O; no non-blocking option for UI/server hosts.

---

5. API Usability Observations

- **Three return conventions coexist.** `Open(string)` → possibly-null Solid; `Open<T>(string, out T)` → bool but throws FileNotFoundException and lets parser exceptions escape; `Open(Stream, string, out TessellatedSolid[])` → bool, swallows all. Parsers themselves disagree (STL/OFF/PLY null, 3MF throws NRE, AMF catch-and-null). Pick one contract, enforce in the shared dispatcher.
- `Open<T>` **conflates "file unreadable" with "wrong solid type"** (`IOFunctions.cs:160–161`).
- **Hidden data reduction on open.** Single-solid Open quietly runs `GetMostSignificantSolid` (`:134–143`), discarding "insignificant" bodies by heuristic (`:178–205`) with only Debug.WriteLine traces. Should be opt-in or surfaced.
- **Format auto-detection extension-only at top level.** Binary STL renamed .ply fails outright; a magic-byte sniffer ("solid", PK, "ply", "OFF", "#") would make `Open(Stream, filename)` robust.
- `TessellatedSolidBuildOptions` **integration uneven.** Null-defaulting in OBJ/AMF/TVGL but not STL/3MF/PLY/OFF. OBJ *changes the number of solids returned* based on `PredefineAllEdges` (`OBJFileData.cs:132`). `ReferenceIndex` doubles as a TVGL-file selector (`TVGLFileData.cs:118–121`) — scope creep.
- **Save API advertises formats it can't write.** `Save(..., FileType.OBJ)` → NotImplementedException (`OBJFileData.cs:426–429` via `IOFunctions.cs:1295–1296`); `SaveToString(solid)` throws with its own default. `CanSave(FileType)`/support matrix or remove dead cases.
- **Positives:** stream-based overloads, FileType-explicit OpenFromString, unit inference, 3MF component/transform recursion (`3MFFileData.cs:185–223`) are good API bones.

Top 5 fixes by impact

1. Invariant-culture parsing/formatting everywhere — currently breaks half the world's locales both directions.
2. `File.OpenWrite` → `File.Create` (`IOFunctions.cs:1270,1351,1435`) — silent file corruption.
3. Route `FileType.OFF` to OFFFileData in all three dispatchers; de-duplicate the dispatch.
4. Fix inverted `Where(string.IsNullOrWhiteSpace)` in all six writers — silent metadata loss.
5. Pad binary STL header to exactly 80 bytes; fix the R/B bit swap.

Appendix G — VoxelizedSolid, MarchingCubes, CrossSectionSolid

Code Review: VoxelizedSolid, MarchingCubes, CrossSectionSolid (TVGL)

All findings verified by reading cited code; call-graph reachability checked by grep where noted.

---

1. Potential Errors

HIGH

1.1 `VoxelRowDense.Subtract(subtrahends, offset)` **calls** `Union` **instead of** `Subtract` — `VoxelRowDense.cs:271-276`. Subtracting from a dense row *adds* the subtrahend's voxels. Latent through public API only because `VoxelizedSolid.Subtract` (`PublicFunctions.cs:155`) forces rows sparse first.

1.2 `VoxelRowDense.Intersect` **with a sparse operand is a silent no-op** — `VoxelRowDense.cs:253-259`. Sparse branch's loop body is an empty statement (`;//this doesn't work IntersectRange(...)`). Even per-range IntersectRange would be wrong — intersection must turn off everything *outside* the ranges (as `VoxelRowSparse.Intersect` at `VoxelRowSparse.cs:312-320` correctly does).

1.3 `VoxelEnumerator.MoveNext` **never visits voxel (0,0,0)** — `VoxelHelperClasses.cs:94-113, 118-121`. First `MoveNext()` immediately increments; `(0,0,0)` never yielded; `Reset()` restores broken state.

1.4 `CrossSectionSolid.Copy()` **destroys the source solid then throws NRE** — `CrossSectionSolid.cs:338`: `Layer2D = new List<Polygon>[NumLayers];` assigns to THIS (the source), then `:341` dereferences null layers. Line should be deleted (ctor at :334 already allocates `solid.Layer2D`).

1.5 `CrossSectionSolid.Reverse()` does nothing — `PublicStaticMethods.cs:216-223`. `newLayers` built/reversed but never assigned back; `_firstIndex`/`_lastIndex` not invalidated; only `StepDistances` replaced → internally inconsistent.

1.6 `CrossSectionSolid.CalculateSurfaceArea()` never assigns `_surfaceArea` — `CrossSectionSolid.cs:532-552`. Computed `area` discarded (compare `VoxelizedSolid.cs:710`).

1.7 `VoxelizedSolid.CalculateCenter()` — wrong formula, missing offset, int overflow — `VoxelizedSolid.cs:659-684`:

- `xTotal += rowCount * voxelRow.AverageXPosition()` — dense `AverageXPosition()` (`VoxelRowDense.cs:368-389`) returns the **sum**, so multiplying by rowCount over-weights; sparse version (`VoxelRowSparse.cs:499-513`) returns **2×** the sum — dense and sparse differ ~2× for same content.
- `_center` in local voxel space (no `Bounds[0]` offset, no half-voxel) while every other Solid reports world coords. Author's `//is this right?` comment.
- `xTotal/yTotal/zTotal` are `int`; overflow for realistic grids (1000^3).
- `Count == 0` → NaN, no guard.

1.8 `MarchingCubesCrossSectionSolid.GenerateOnLayers` — ring-buffer off-by-one gives every cube a stale top layer — `MarchingCubesCrossSectionSolid.cs:141-148` + `GetValueFromSolid` :576-581. At iteration k, `MakeFacesInCube(i, j, k)` reads z = k+1 → slot `(k+1)%2` = layer k−1, not k+1. `GetValue` (`MarchingCubes.Base.cs:255-275`) caches the wrong values permanently. Should be `MakeFacesInCube(i, j, k - 1)`.

1.9 Inconsistent `yStartIndex` conventions for `AllPolygonIntersectionPointsAlongHorizontalLines` — implementation (`PolygonOperations.IsInside.cs:699-756`) appends an entry for **every** step from `startingYValue` (contradicting its own doc :691).

- `FillInFromTessellation` (`VoxelizedSolid.cs:270-276`) matches. ✓
- `CreateFrom(polygons…)` (`VoxelizedSolid.cs:232-241`) shifts rows by `yStartIndex` and doesn't clamp `yStartIndex + j` → misregistration + potential IndexOutOfRange. ✗
- `MarchingCubesCrossSectionSolid.CreateDistanceGrid` (:262-269) assumes list omits leading empties → every grid row gets the intersections of the row below. ✗

1.10 `BooleanOperation(PrimitiveSurface, …)` drops the trailing range — `VoxelizedSolid.Advanced.cs:126-150`. Odd crossings count with `startDefined == true` → final `[start, numVoxelsX)` never applied (multi-surface overload handles it, :278). `Subtract(plane)` can be a no-op on most rows; for `Intersect` (inverseRange), rows outside the surface bbox and zero-crossing rows (:121) never cleared → AND semantics wrong beyond bbox.

1.11 Multi-surface `BooleanOperation` inserts a world coordinate into a list of line parameters — `Advanced.cs:269-278`. Head sentinel `ConvertXIndexToCoord(0)` = world coord, then `XMin` added again at :276. Should insert `0.0`.

1.12 `SliceOnPlane(Plane)` returns halves swapped when `plane.Normal.X < 0` — `PublicFunctions.cs:265-289`. X-crossing branch always assigns upper-x side to vs1 regardless of normal sign (x-negligible branch :252-264 handles sign correctly).

1.13 `MinkowskiSubtractOne` never flushes the last row's pending ranges — `Advanced.cs:44-76`. Ranges applied only when (y,z) changes; final row's accumulated ranges silently dropped.

MEDIUM

1.14 `MarchingCubes.Implicit` seed functions inconsistently populate `cellIDsToCheck` — `FindZPointFromXandY` adds own hashID in both branches (:242, :278, :306); `FindYPointFromXandZ` omits it in two-intersection branch (:393-405, :419-432); `FindXPointFromYandZ` omits in **both** (:480-492, :515-527, :541-554) → holes in generated mesh.

1.15 Single-surface `BooleanOperation` excludes the last row/slab of the surface bbox — `Advanced.cs:105-114`: `j < maxJ` with inclusive maxIndices (multi-surface uses inclusive, :187, :195).

1.16 Both `BooleanOperation`s hard-cast rows to `VoxelRowSparse` without converting — `Advanced.cs:117, 218`. After `UpdateToAllDense()` → `InvalidCastException` inside `Parallel.For`.

1.17 `VoxelRowDense.Invert` flips padding bits — `VoxelRowDense.cs:347-352`; `numBytes = 1 + (numVoxelsX >> 3)` always leaves 1–8 spare bits → phantom voxels in Count/AverageXPosition/XIndices/GetDenseRowAsSparseIndices. Latent (public Invert forces sparse first). `GetNeighbors` (:84-101) ignores `upperLimit` — reads padding bit as neighbor.

1.18 Sparse indexer `get` is unlocked while `BinarySearch` mutates shared state — `VoxelRowSparse.cs:71-75` (lock commented out); `lastIndex` cache (:35, :184, :191, :202) written by every search. Concurrent reads race; reader racing writer can see List mid-Insert → wrong answers or exceptions. Every other member locks; the getter is the hole.

1.19 `CreateFrom(IEnumerable<Polygon>, …)` writes raw unvalidated sparse indices — `VoxelizedSolid.cs:236-242`. No `sp == ep` skip, no merge on touching ranges (compare :281-286) → violates strictly-increasing invariant BinarySearch relies on.

1.20 `GetExposedVoxels` vs `GetExposedVoxelsWithSides` disagree about x-boundary voxels — `VoxelizedSolid.cs:744` vs `:788`. Two enumerations can disagree; `CalculateSurfaceArea` (:698-711) uses the latter.

1.21 `VoxelRowSparse.XIndices` can overrun `indices` and return values below `start` — `VoxelRowSparse.cs:526-528`. Dense counterpart worse: `xValue = start >> 3` (byte count as x coord) yet enumerates from `values[0]` (`VoxelRowDense.cs:391-422`). Latent — all call sites use start=0.

1.22 `GetCrossSectionsAs3DLoops` NREs on empty (null) layers — `PublicStaticMethods.cs:126-146` (lazy allocation gaps per `CrossSectionSolid.cs:179-183`). Same `GetTotalPolygonVertices` (`CrossSectionSolid.cs:567-573`).

LOW

1.23 `MarchingCubesDenseVoxels.GetOffset` places the crossing one voxel early — `MarchingCubes.VoxelizedSolid.cs:102-127` (offset `…*i` instead of `…*(i+1)`); systematic surface bias. **1.24 `ConvertXCoordToIndex` clamps only the low side** — `VoxelizedSolid.cs:1019-1043`; unchecked double→ushort above bounds (unspecified value). **1.25 `VoxelizedSolid.Equals` no null-check; ignores `VoxelSideLength`/`Bounds`** — `VoxelizedSolid.cs:1102-1117`. **1.26 `numVoxelsY * numVoxelsZ` products overflow int for extreme grids** — `VoxelizedSolid.cs:127, 164`; individually range-checked (:383-389), product not. **1.27 `MarchingCubesCrossSectionSolid` ctor** — divides by `NumLayers - 1` (:80); `while` overruns `Layer2D` if all layers empty (:93/:96).

---

2. Redundancies

- **MEDIUM — `MarchingCubesImplicit.MakeFacesInCube` ~60-line near-verbatim copy of base** (`Implicit.cs:122-198` vs `Base.cs:283-350`); only delta is `Coordinates.IsNull()` guard — hoistable.
- **MEDIUM — `FindZPointFromXandY`/`FindYPointFromXandZ`/`FindXPointFromYandZ` are three axis-permuted copies** of the same ~120-line routine (`Implicit.cs:202-570`); divergence already caused bug 1.14.
- **MEDIUM — dead/vestigial code:** `CreateDistanceGridBruteForce` (`MarchingCubesCrossSectionSolid.cs:195-251`, no callers; its Parallel.For also drops the i == iMax column); `GenerateBetweenLayers` unreachable after throw (:165-188); `ConvertToLoftedTessellatedSolid` body commented out, returns empty solid (`CrossSectionSolid.cs:262-294`); `EdgeDirectionTable` unused (`Base.cs:416-421`); `comments` list built and discarded (`Base.cs:208-211` and MCCSS :153-157, 183-187); commented-out TurnOff mirror (`VoxelRowDense.cs:130-137`); unused usings (`PublicFunctions.cs:18-21`); `IncreasingDoublesBinarySearch` self-documented as redundant with List.BinarySearch (`Advanced.cs:291-315`).
- **LOW — four CreateFrom/CreateEmpty/CreateFullBlock ctors repeat the same 10-line init block** (`VoxelizedSolid.cs:150-206, 313-334, 341-381`); divergence already: one copies `ts.SolidColor`, other uses `Constants.DefaultColor` (:158 vs :193).
- **LOW — `VoxelRowDense.Count` and `AverageXPosition` duplicate the same 8-branch bit ladder** (:147-159, :374-386) — collapse under PopCount.

---

3. Inefficiencies

HIGH

3.1 Per-voxel virtual + lock cost in hot loops. `Draft` Y/Z directions (`PublicFunctions.cs:347-392`) set voxels one at a time through public indexer: bounds-check → virtual → lock → binary search + List.Insert = O(n^3) locked virtual calls; per-row range ops (as X-direction branches do, :323-346) would be O(rows). Same pattern: `VoxelEnumerator.MoveNext` (`VoxelHelperClasses.cs:110`, indexer call per grid cell incl. off-voxels), neighbor probes in `GetExposedVoxels*` (`VoxelizedSolid.cs:746-749, 797-800`).

3.2 Bit counting without `BitOperations`. `VoxelRowDense.Count` (:143-162) tests 8 bits/byte in branches; `BitOperations.PopCount` over `MemoryMarshal.Cast<byte, ulong>(values)` ~10× faster. Same for AverageXPosition (TrailingZeroCount) and `GetDenseRowAsSparseIndices` (`VoxelizedSolid.cs:440-460`).

3.3 MarchingCubes valueDictionary grows to the full grid; every cube allocates. `GetValue` caches every corner forever (`Base.cs:255-275`, eviction commented out) → numGridX·Y·Z heap StoredValue objects when only two z-slabs are needed. `MakeFacesInCube` allocates `StoredValue[8]`, `Vertex[12]`, `Vertex[3]` per cube (:291, :309, :340) — reusable scratch given sequential loop.

MEDIUM

3.4 LINQ allocation per row inside Parallel.For — `PublicFunctions.cs:93, 128, 161`: `solids.Select(s => s.voxels[i]).ToArray()` per row. **3.5 `VoxelRowSparse.Union/Intersect/Subtract` copy dense operands per row-op** — `VoxelRowSparse.cs:288, 309, 337` via yield-based enumerator allocating a List each time; in practice unreachable (everything pre-converted). **3.6 Missed/misused Parallel:**

- `MarchingCubes<>.Generate` (`Base.cs:204-207`) serial triple loop; z-slabs parallelizable with per-slab dictionaries.
- `FillInFromTessellation` (`VoxelizedSolid.cs:262-263`) has its Parallel.For commented out though each k writes disjoint rows — single most expensive voxelization step, trivially parallel.
- `GetExposedVoxelsWithSides` abandoned BlockingCollection parallel version commented out (:758-772). **3.7 `SliceOnPlane` copies the whole solid twice then clears half of each** — `PublicFunctions.cs:185-186, 243-244`. `Copy()` itself (`VoxelizedSolid.cs:116-135`) always converts to sparse and calls full `UpdateProperties()` recompute. **3.8 Span/ArrayPool:** dense boolean kernels (`VoxelRowDense.cs:250-251, 290-291, 330-331`) byte-at-a-time → Span 8× fewer iterations. Distance grids per layer in `CreateDistanceGrid` (:261) perfect ArrayPool candidates (only 2–4 alive at once).

LOW

3.9 Row-major layout (x-runs indexed y + numVoxelsY·z) good for z/y probes, bad for Draft-Y sweeps — document for contributors. **3.10 `lock` held across `yield return`** — `VoxelRowDense.XIndices` (:394), `VoxelRowSparse.XIndices` (:523). Monitor held for enumeration lifetime; abandoning enumerator without dispose leaks the scope.

---

4. Other Concerns

- **Thread-safety undocumented and inconsistent (MEDIUM).** Dense rows lock on `values`, sparse on `indices`, but sparse getter unlocked (1.18); solid-level ops swap row objects unsynchronized (`VoxelizedSolid.cs:405, 419`); Count/_center updated non-atomically. Either document "not thread-safe; locks only protect built-in Parallel loops" or remove per-row locks (they cost every indexer call).
- **Memory footprint undocumented (LOW).** `voxels` = numY·numZ object refs + per-row List overhead even for empty rows — 2048^2 y/z grid ≈ 350 MB before voxel data.
- **ushort limits (LOW).** 65535/axis enforced with plain `Exception` (`VoxelizedSolid.cs:387`); `ushort.MaxValue` used as sentinel (:843-844) makes a true 65535-wide grid ambiguous; y·z product can overflow int.
- **Tolerance handling (LOW).** `ToleranceForSnappingToLayers = 0.517` (`MarchingCubesCrossSectionSolid.cs:37`) > 0.5 makes `onLayers` true for every discretization → `GenerateBetweenLayers` unreachable, snapping test vacuous. `ExpandHorizontally/Vertically` hard-code 1e-9 (:327, :407). `MarchingCubesImplicit.IsInside` uses `<=` while `GetOffset` uses `IsPracticallySame` (:74, :88-90) — two different boundary tolerances.
- **`VoxelizedSolid.Transform`/`TransformToNewSolid` and `CrossSectionSolid` equivalents throw `NotImplementedException`** (`VoxelizedSolid.cs:621-635`, `CrossSectionSolid.cs:358-373`) — runtime landmines for generic Solid consumers.

---

5. API Usability Observations

- **Empty public no-op methods ship as API (HIGH).** `MinkowskiAddBlock/MinkowskiSubtractBlock/MinkowskiAddSphere/MinkowskiSubtractSphere` (`Advanced.cs:27-34`) have empty bodies — silent success. Implement, throw, or delete.
- **Construction API asymmetric (MEDIUM).** `CreateFrom(TessellatedSolid, int|double, bounds)` discoverable and well documented; no from-function/implicit-field constructor; `CreateFrom(polygons, …)` requires mandatory bounds; the two TS overloads silently differ on SolidColor.
- **Conversion round-trips (MEDIUM).** `ConvertToTessellatedSolidMarchingCubes(int voxelsPerTriangleSpacing)` vs `CrossSectionSolid.ConvertToTessellatedSolidMarchingCubes(double gridSize)` vs approximate-triangle-count overload — different parameter conventions; shared options type would help. `ConvertToTessellatedExtrusions(bool extrudeBack, bool createFullVersion)` ignores `createFullVersion` (`CrossSectionSolid.cs:203-206`).

- **Indexer semantics (MEDIUM).** get returns false out of bounds but set throws (`VoxelizedSolid.cs:838-879`) — undocumented asymmetry; internal sparse sentinel leaks into public contract (:842-845). `ConvertXIndexToCoord` docs say "lower bound" but returns center (+0.5, :1044-1063).
- **Naming/discoverability (LOW).** Public camelCase `numVoxelsX/Y/Z`; `VoxelsPerSide` allocates per read (:48); `AverageXPosition` returns a sum (×2 sparse); `FractionDense` only ever 0 or 1; `GetLongID/GetIndicesFromID` (:565-572) public undocumented 21-bit packers.
- **`CrossSectionSolid.Layer2D` is a public mutable field** (`CrossSectionSolid.cs:50`) with cached `_firstIndex/_lastIndex` only `Add` maintains (:188-189) — direct assignment (which the class itself does, bug 1.4) desyncs the cache.

Appendix H — Miscellaneous Functions & Top-Level Classes

Code Review: TVGL Miscellaneous Functions + Top-Level Files

Scope: All files in `Miscellaneous Functions\` plus listed top-level files. All files read in full; every finding verified against source.

---

1. Potential Errors

High

- **`MiscFunctions.cs:2165` — operator-precedence bug in `SkewedLineIntersection`.** `center = intersect1 + intersect2 / 2;` computes `intersect1 + (intersect2/2)` instead of the midpoint. Every caller requesting the "middling point" of two skew lines gets a wrong point.

- **`MiscFunctions.cs:244-255` — `GetMinVertexDistanceAlongVector` never increments the index counter.** `var i = 0` declared but (unlike the max version at line 221) never incremented; returned `index` always 0 (or -1 for empty input).

- **`MiscFunctions.cs:2987-2993` — `Get3DLineValuesFromUnique` duplicated nested condition makes the −Z branch unreachable.** `if (direction.Z > 0) { if (direction.Z > 0) ... }` — for −Z lines execution falls through to line 2996 where a near-zero vector is normalized → NaN/garbage axes. Breaks round-trip with `Unique3DLine` (line 2933) and corrupts `Unique3DLineHashLikeCollection.AddIfNotPresent` (:256) and `FindBestRotations` for downward axes.

- **`SphericalHashLikeCollection.cs:320` — `radii` inserted at index 0 instead of `i`.** Misaligns the parallel lists; `IsTheSame` (:497, `radii[existingIndex]`) compares wrong radius when `ignoreRadius == false`. Generic subclass (:67–70) does it correctly, confirming typo.

- **`UpdatablePriorityQueue.cs:278-314, 379-406` — `DequeueEnqueue`/`EnqueueDequeue` leak stale `_elementIndices` entries.** Never remove the root's dictionary entry → `Contains(removedRoot)` true, `Remove` sifts wrong slot (heap corruption), `_elementIndices.Add(element, 0)` (:296, :308) throws if element ever enqueued before.

- **`UpdatablePriorityQueue.cs:566-584` — `Remove` restores heap by sift-down only — does not preserve heap invariant.** When the relocated last node's priority is smaller than the removed slot's parent, the min-heap property is violated upward. Matters greatly: `UpdatePriority` (:967–971) is Remove+Enqueue, used pervasively in `PolygonOperations.Simplify.cs` (:204, 265–266, 743–744, 1274–1355) and `SimplifyTessellation.cs:163-164`. Fix: compare with parent; MoveUp when smaller else MoveDown.

- **`UpdatablePriorityQueue.cs:430-435` — `EnqueueRange(ICollection)` builds `_elementIndices` from the whole backing array, not the first `count` slots.** Trailing `default` slots → `ArgumentNullException` (ref types) or duplicate-key `ArgumentException` (value types).

- **`Proximity.cs:521-526` — `ReduceDirections` reduction loop is dead code.** `sortedProximities` uses `NoEqualSort` (never returns 0) so `ContainsKey` always false (also passes an *index* as a *key*); loop exits first iteration — `targetNumberOfDirections` never honored. Lines 553/559 test identical condition in `if` and `else if`.

- **`MiscFunctions.cs:1097-1099` — `GetMultipleSolids` creates a primitive copy and discards it.** `primCopy` computed (incl. fragile O(n²) `faceGroup.IndexOf(f)` mapping) but `newSolid.Primitives.Add(primitive)` adds the *original* wired to the old solid. `// does it need to be copied?` comment confirms confusion.

- `SphericalHashLikeCollection.cs:36–48, 75–81, 192–207` **(and** `Unique3DLineHashLikeCollection.cs:36–49, 73–79, 147–161`**) — method hiding via** `new` **breaks the parallel** `items` **list.** Base implements `ICollection<T>`; calls through base-typed reference add to `sphericals/cartesians/radii` **without** adding to `items` → silent desync. Needs virtual methods or composition.

Medium

- `SphericalAnglePair.cs:101–118` **— GetHashCode/Equals contract violation.** Equals is tolerance-based (dot ≈ 1 within 5e-8); GetHashCode quantizes into π/32768 buckets — near-equal pairs straddling a boundary hash differently → Dictionary/HashSet misses. Tolerance-based Equals also intransitive. `azimuthInt` −π special case maps far from −π+ε.
- `SphericalHashLikeCollection.cs:188` **—** `TryGet(..., out T, out SphericalAnglePair)` **indexes** `sphericals[i]` **on the failure path** — `i == Count` for misses at end → `ArgumentOutOfRangeException`; throws on empty collection.
- `MiscFunctions.cs:1503–1547` **—** `TransformToXYPlaneMaybeBetter` **±Y special case returns a reflection (det −1), not a rotation**; geometry mirrored; forward/back transforms set to same matrix. No callers currently, but public.
- `MiscFunctions.cs:2658–2675` **—** `IsVertexInsideTriangle(IList<Vector3>, …)` **ignores** `onBoundaryIsInside`**; boundary handling internally inconsistent** (`< 0` vs `> 0`). Same asymmetry in TriangleFace overload (:2624, 2628); `PointOnTriangleFromRay` (:2324–2331) never forwards the flag.
- `VolumeCenterMoments.cs:34–80` **—** `CalculateVolumeAndCenterOLD` **accumulates across refinement iterations without resetting** — still public; results ~3× too large. Delete or `[Obsolete]`.
- `VolumeCenterMoments.cs:121–123` **— division by** `volume == 0` **mid-refinement → NaN center**, which then poisons the next pass and skips the `IsNegligible` fallback (:126).
- `Comparators.cs:33–38` **—** `SortByIndexInList` **inconsistent comparer** (both orderings return 1 for equal indices). `TwoDSortXFirst`/`TwoDSortYFirst` (162–167, 184–189) never return 0 while using tolerant `IsPracticallySame` on the primary key. `NoEqualSort` (136–140) deliberately never-equal — but that breaks `ContainsKey` (see ReduceDirections) and violates `Compare(x,x)==0`.
- `MiscFunctions.cs:154–155 / 181–182` **— unbounded decimal-digit loop** — `numDecimalPoints` can exceed 15 → `Math.Round` throws. Siblings at 1231/1299 clamp correctly.
- `OutputServices.cs:42–54` **—** `SetLogger` **disposes the** `ILoggerFactory` **before the logger is used** (`using` factory); subsequent `Log.*` writes through disposed provider. Lazy static init unsynchronized.
- `Presenter.cs:182–190, 267–280` **—** `EmptyPresenter2D` **throws** `NotImplementedException` **for** `SaveToPng` **and all** `ShowStepsAndHang` **overloads** while other members are silent no-ops. Null-object default crashes.
- `MiscFunctions.cs:737–784` **—** `FindFlats` **off-by-one: seed face excluded from** `area` **and** `numFaces` — `minNumberOfFacesPerFlat = 2` actually requires 3 faces.
- `UpdatablePriorityQueue.cs:207–231` **—** `Enqueue` **does not enforce documented uniqueness invariant** — duplicate enqueue silently corrupts index tracking.
- `SolidAssembly.cs:618–640` **—** `GetAllFilePathsRecursive` **adds** `null` **paths; skip test inverted** — `if (skipEmbeddedFiles && filePath != null && File.Exists(filePath)) continue;` *skips files that exist* and collects null/missing ones; `Path.GetFileName(null)` downstream (:613).
- `Proximity.cs:845–851` **—** `FindBestRotations` **relies on** `TryGet`**'s failure side-effect** (out param left as the reflected query, `Unique3DLineHashLikeCollection.cs:321`); works only because Reflect is idempotent. Use `uniqueLine` directly.

Low

- `MiscFunctions.cs:301` — tuple element misnamed (`minPoint` for max).
- `MiscFunctions.cs:1840` — asymmetric interval test (`t_a > 1` vs `t_b >= 1`) in `SegmentSegment2DIntersectionConventional`.
- `MiscFunctions.cs:454–456` — `DefineInnerOuterEdges` enumerates `faces` three times.
- `MiscFunctions.cs:2751–2783` — `SetPositiveAndNegativeShifts` recursive with no depth bound.
- `MiscFunctions.cs:2686–2697` — `IsPointInsideSolid` hardcodes 1e-8 instead of `ts.SameTolerance`; slices exactly at `PointInQuestion.Z` (degenerate on facet planes).
- `SphericalHashLikeCollection.cs:51–57 / 303–309` — second `Reflect` call is a no-op, misleading.
- `Proximity.cs:783–785` — `NEquidistantSpherePointsKogan(1)` divides by zero.
- `Proximity.cs:16` — stray `using System.Drawing;`. `Presenter.cs:3` — stray `using static ...JSType;`.
- `Colors.cs:837–866` — `GetRandomColorNames`/`GetRandomColors` skip family 0 after first cycle; intentionally infinite enumerables (foreach without Take hangs).
- `Colors.cs:741–805` — `Distinct64Colors` contains duplicates (DodgerBlue ×2, HotPink ×2) → only 62 distinct.
- `Colors.cs:698–712` — `GetHue` NaN for achromatic colors; comment wrong (result 0..1, not degrees).

- `MiscFunctions.cs:2823–2857` — `OrderedEdgesAndAnglesCCWAtVertex` infinite-loops/NREs on open border fans.
- `SolidAssembly.cs:130–139` — ctor NREs on null `fileName`/`Name`; `IsEmpty()` (:181) NREs if `Solids` never set.
- `Constants.cs:113–125` — doc comments missing `</summary>`; `CartesianDirections` (486–533) garbled nested summaries.

---

2. Redundancies

High

- **`MiscFunctions` is a 3-file partial god-class** — MiscFunctions.cs (3104 lines) + Proximity.cs (887) + VolumeCenterMoments.cs (194) ≈ 4200 lines, ~150 public methods mixing sorting, projection, transforms, angles, intersections, distances, point-in-X, mesh topology, reflection-based type discovery, sphere sampling, volume/moments.
- **Three implementations of "transform direction to XY plane":** `TransformToXYPlane` (1455–1490), `TransformToXYPlaneMaybeBetter` (1503–1547 — zero callers, buggy), `SphericalAnglePair.TransformToXYPlane` (:85–92).
- **Four point↔segment/line distance routines:** `MiscFunctions.DistancePointToLine` (2185–2229), `Proximity.ClosestVertexOnSegmentToVertex` (149–168), `Proximity.ClosestPointOnLineSegmentToPoint` (181–243), `PolygonOperations.SqDistancePointToLineSegment` (`PolygonOperations.DistanceToPoint.cs:187`).
- **Dead/legacy public methods:** `CalculateVolumeAndCenterOLD`; `TransformToXYPlaneMaybeBetter`; `CalculateInertiaTensor` (VolumeCenterMoments.cs:153–192 — full pass of work then unconditional `throw new NotImplementedException()` at :191); `OrderedFacesCCWAtVertexNoEdges` (MiscFunctions.cs:2914–2917, throws); `SubAssembly.Transform`/`TransformToNewAssembly`/`CalculateCenter`/`CalculateInertiaTensor` (SolidAssembly.cs:726–758, all throw).

Medium

- **Copy-pasted Vertex vs Vector3 overload bodies:** `AreaOf3DPolygon` (803–877 vs 892–966 — 75 identical lines), `GetVertexDistances` (149–163 vs 176–190), `Length` (601–627), `ConvertTo1DDoublesCollection` (2790–2815).
- **`ChooseTightestLeftTurn` duplicated verbatim** at Proximity.cs:696–713 and 722–739.
- **Two 2D segment-segment intersectors:** conventional (1805) + PGA-based (1863); doc on the first advertises the second.
- **`GetOrthogonalDirection` pure alias of `GetPerpendicularDirection`** (Proximity.cs:329–330).
- **`Length(polyline, isClosed:true)` duplicates `Perimeter`** (573–608); `isClosed = true` a surprising default.
- **`Comparators.ForwardSort`** reimplements `Comparer<double>.Default`; none of the comparers handle NaN coherently.
- **Commented-out blocks:** UpdatablePriorityQueue `FindIndex` ~28 dead lines (912–939), stale remarks (546–551, and ignored `equalityComparer` param :556); SolidAssembly.StreamRead 17-line alt (242–259); MiscFunctions 1461–1465, 1622–1624, 1836–1837.
- **`Global.FindIndex`** (Global.cs:21–33) needless `ToList()`.

UpdatablePriorityQueue vs .NET PriorityQueue

Mostly not redundant: it's a ~900-line fork of `System.Collections.Generic.PriorityQueue` (acknowledged :12–13) whose value-add is O(1) `Contains`/`Remove`/`UpdatePriority` via `_elementIndices`. .NET's built-in `Remove` is O(n). Recommendation: keep but (a) fix invariant bugs, (b) delete forked dead code, (c) make `UpdatePriority` O(log n) single-sift, (d) add behavioral tests vs PriorityQueue.

---

3. Inefficiencies

Medium

- **`FacesWithDistinctNormals` (699–727):** three full `OrderBy().ToList()` sorts + `RemoveAt(i)` in reverse loop — $O(n^2)$ per pass.
- **`FindFlats` (768):** `Plane.DefineNormalAndDistanceFromVertices` re-fits over the *entire accumulated vertex list per candidate edge* — $O(k^2)$ per flat. Incremental centroid/covariance update = O(1) per test. `stack.Any()` (752) allocates per iteration.
- **`PointOnTriangleFromLineSegment` (2284):** allocates `List<Vector3>` per call in ray-cast inner loop.
- **`Proximity.FindBestPlanarCurve` (589–601):** reflection-driven — `TypesImplementingICurve()` re-scans assembly types per call (MiscFunctions.cs:3007–3014, no caching); `MethodInfo.Invoke` boxing. Cache + compiled delegates or static-abstract dispatch.
- **`ProjectTo2DCoordinates*` (1226–1318):** two `Math.Round` per key; scaled-integer key cheaper.
- **`SphericalHashLikeCollection`/`Unique3DLineHashLikeCollection` insertion O(n) per add** (List.Insert into 4 parallel lists) → $O(n^2)$ build. "Hash-like" oversells: tolerance-aware sorted list. Bucketed design (quantized polar bands) → near-O(1). Four

parallel Lists → one List for locality.

- `CalculateVolumeAndCenter` **(100–125):** enumerates `faces` up to 10× (once per refinement); materialize once. Same multiple-enumeration in UpdatablePriorityQueue items ctor (171–174).
- `TriangleFace.Edges` **is a compiler-generated iterator** (`TriangleFace.cs:327-335`) — every `foreach (var edge in face.Edges)` allocates. Mesh-wide inner loops (FindFlats 758, GetContiguousFaceGroups 1149, DefineInnerOuterEdges 462…). Exposing AB/BC/CA or struct enumerator = cheap library-wide win.
- **Boxing:** `SphericalHashLikeCollection<T>.Contains` (217), `Unique3DLineHashLikeCollection<T>.Contains` (170) — `item.Equals(items[i])` on unconstrained T; use `EqualityComparer<T>.Default`.

Low

- `ClosestVertexOnTriangleToVertex` computes `Distance` (sqrt) up to twice (48, 60, 67).
- `SortAlongDirection` doc describes O(n) counting-sort that no longer exists (90–97).
- `DebugUtilities.SaveBitmap` (229): two passes for Max/Min; ZRange zero-guard missing.
- Parallel opportunities: per-face loops in `CalculateVolumeAndCenter` (sum reduction), `FacesWithDistinctNormals`, `GetContiguousFaceGroups` seeding — nothing uses TPL today.
- Span/stackalloc: `PointCommonToThreePlanes` (2011–2015) allocates `double[,]` + `double[]` per call for a 3×3 solve.

---

4. Other Concerns

- **High — god-class discoverability:** `MiscFunctions.` IntelliSense is the de-facto index of the geometry toolbox; doc-comment "Common search terms" (36, 73) is a symptom.
- **High — static mutable service locator (`OutputServices` + `Presenter` + `Log`):** mutable static ILogger/IPresenter with lazy unsynchronized init (OutputServices.cs:7–40); 40-method static façade; tests order-dependent; two hosts can't use different sinks. Fix disposed-factory bug; `Interlocked`/`Lazy<T>`; accept `ILoggerFactory`.
- **Medium — mutable public static collections:** `Color.ColorDictionary` (Colors.cs:884, mutable dictionary of *mutable* Color objects with public byte fields, :627–644), `Distinct64Colors` (741), `PlatonicConstants.*Directions` (154–340, cached arrays returned by reference — caller writes corrupt future callers; benign lazy-init race), `Constants.TessellationToVoxelizationIntersectionCombinations` (177–187, writable list+arrays). Return ImmutableArray/ReadOnlySpan or freeze.
- **Constants design:** all const/static readonly — no mutable global tolerances. Real issue: fixed absolute tolerances (BaseTolerance 1E-9 :83; PolygonSameTolerance 1e-7 :94) baked into defaults regardless of model scale; `Solid.SameTolerance` exists (Solid.cs:254) but misc functions never consult it.
- **Logging design:** `Log` is internal (Logger.cs:6) so clients can't route through it; first stray `Log.Trace` before host configuration permanently binds broken console logger; `Log.BeginScope` discards the IDisposable.
- **Naming (Low):** `DodechedronDirections` (PlatonicConstants.cs:228); `TransformToXYPlaneMaybeBetter`; `CalculateVolumeAndCenterOLD`; "HashLike" for sorted-list; `oneOverdeterminnant` (1833); `FileType.ThreeMF` documented as "Mobile MultiModal Framework" (Constants.cs:337); `SolidAssembly._distinctSolids` underscore-named property (:98).
- **Dead/debug scaffolding:** `BuildAssemblyTreeNode var debug = false;` + Console.WriteLine blocks (SolidAssembly.cs:328–364), returns `dynamic` (anonymous types); `useOnSerialization` (270–297) never true → serialization hooks dead.
- `SolidAssembly.cs:20` **—** `using System.Numerics;` shadowed by TVGL's own Matrix4x4 — invites confusion.

---

5. API Usability Observations

Reorganizing MiscFunctions (proposed split, keeping extension-method ergonomics)

New static class	Members (current locations)
`DirectionalSortExtensions`	SortAlongDirection, GetVertexDistances, GetMin/Max…AlongVector (62–435)
`PlanarProjection`	ProjectTo2DCoordinates*, ConvertTo2D/3DLocation(s), TransformToXYPlane (1191–1582)
`AngleFunctions`	all Angle*/*AngleBetween* (1585–1782)
`IntersectionFunctions`	SegmentSegment2D*, SegmentLine2D, LineLine2D, plane/three-plane/skew-line, PointOnPlaneFrom*, PointOnTriangleFrom* (1786–2580)
`ProximityFunctions`	Proximity.cs + DistancePointToLine/Plane (2171–2259)

New static class	Members (current locations)
`ContainmentFunctions`	IsVertexInsideTriangle, IsPointInsideSolid, IsPointInsideAABB (2607–2740)
`MeshTopologyFunctions`	DefineInnerOuterEdges, GetVertices, GetMultipleSolids, GetContiguousFaceGroups, FindFlats, FacesWithDistinctNormals, OrderedEdges/FacesCCWAtVertex (438–1189, 2817–2917)
`VolumeAndMoments`	VolumeCenterMoments.cs (minus OLD)
`Line3DDescriptor` struct	Unique3DLine/Get3DLineValuesFromUnique (2928–3001) — a `readonly struct` with Encode/Decode/Reflect would give the hash-like collection a typed key instead of bare Vector4 (Z-is-polar-angle is tribal knowledge)

Also: delete *OLD/MaybeBetter; make helper-only members private/internal; `[EditorBrowsable(Never)]` on reflective `TypesImplementingICurve`/`TypesInheritedFromPrimitiveSurface` (3007–3026).

Solid base class (Solid.cs)

- `SurfaceArea` silent no-op setter (:120). Public setters on Volume/Center desync caches; prefer protected + `InvalidateCaches()`.
- `Bounds` settable public `Vector3[2]` initialized to zeros → XMin..ZMax return 0 rather than "not computed" (161–192).
- Copy ctor (302–325) shallow-copies ConvexHull/SolidColor, doesn't copy Primitives — undocumented contract.
- `InertiaTensor` get-only property backed by `CalculateInertiaTensor` that throws (VolumeCenterMoments.cs:191) — landmine.

SolidAssembly / SubAssembly

- Two-phase construction (Add → `CompleteInitialization()`, 150–157) easy to misuse; forgetting leaves `Solids` null; adding after silently desyncs. Builder or lazy derivation.
- `GetTessellatedSolids(out IEnumerable, out IEnumerable)` (167–171) returns lazy queries via out params; XML doc references removed params.
- `AllPartsInGlobalCoordinateSystem` (558–566) caches lazy IEnumerable that re-transforms per enumeration; first enumeration pays full mesh-copy cost invisibly.
- `BuildAssemblyTreeNode` returns `dynamic` (326) — consumers can only use via JSON round-trip; small DTO needed.
- `numberOfSolidBodies`/`numberOfSheetBodies` caches (71, 85) never invalidate when Solids reassigned.

Hash-like collections

Should be sealed types *containing* a sorted structure rather than inheritance + new-hiding. `AsAnglePairs()`/`Vector3s` (276–284) expose live internal lists — return read-only views.

Top 10 actionable fixes (by impact)

1. UpdatablePriorityQueue.Remove sift-up-or-down + DequeueEnqueue/EnqueueDequeue dictionary removal.
2. Get3DLineValuesFromUnique −Z branch.
3. radii.Insert(0, …) → Insert(i, …).
4. SkewedLineIntersection midpoint precedence.
5. GetMinVertexDistanceAlongVector missing i++.
6. EnqueueRange dictionary-from-full-array.
7. ReduceDirections dead loop / NoEqualSort.ContainsKey.
8. GetMultipleSolids discarded primCopy.
9. OutputServices.SetLogger disposed factory; EmptyPresenter2D throwing members.
10. Remove/obsolete dead public trio: CalculateVolumeAndCenterOLD, TransformToXYPlaneMaybeBetter, CalculateInertiaTensor.